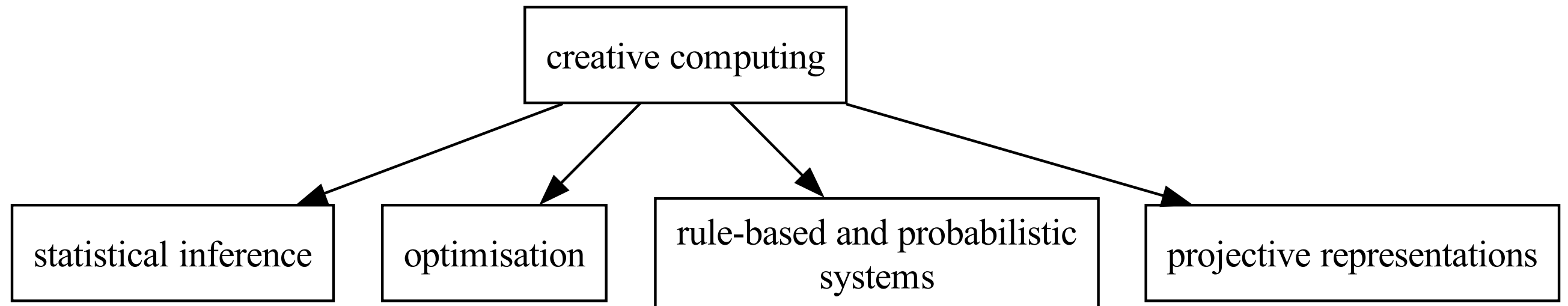




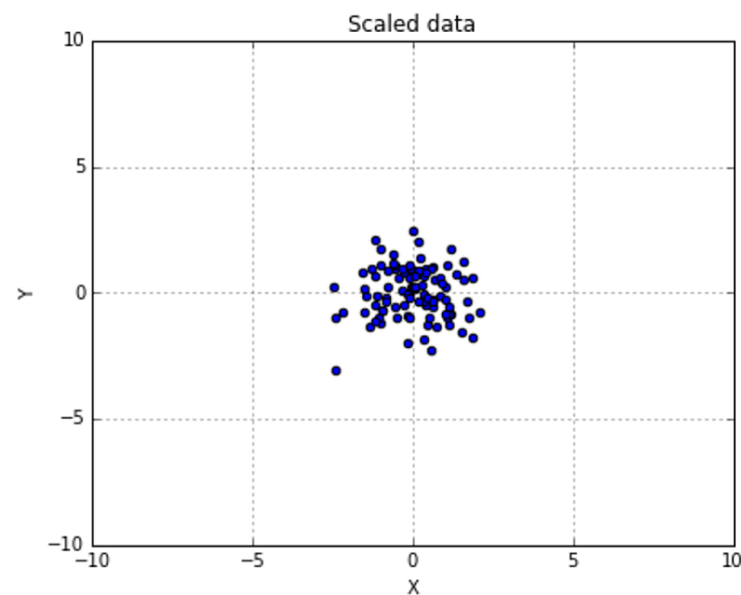
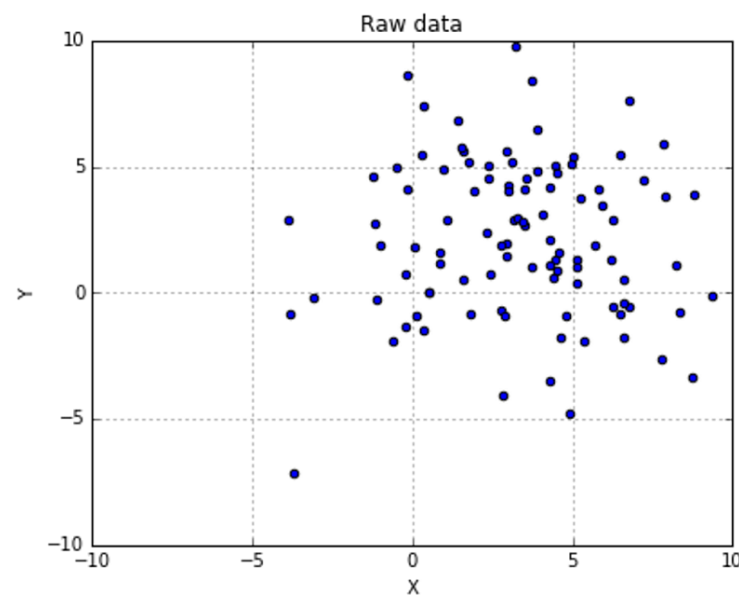
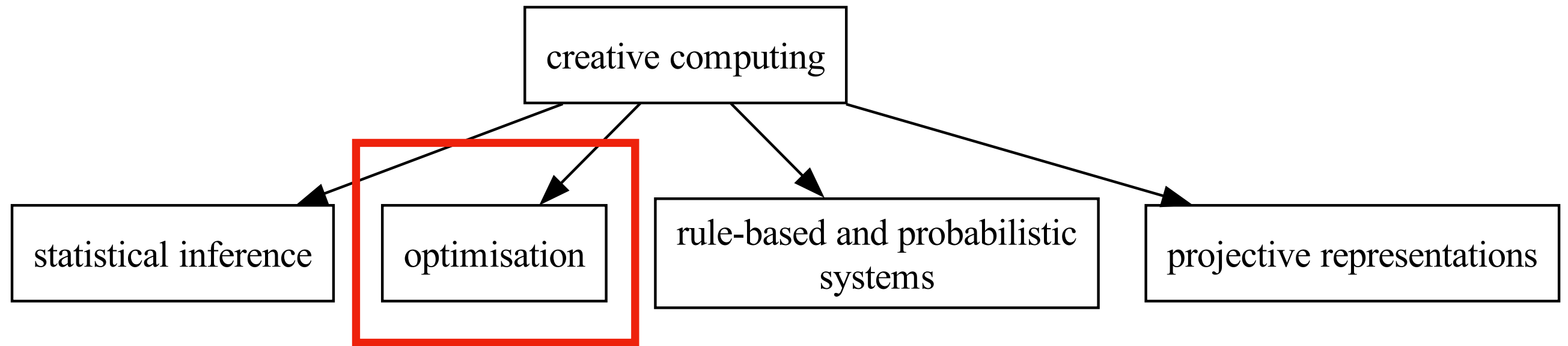
CARMINE-EMANUELE CELLA

INTRODUCTION TO OPTIMISATION AND UNSUPERVISED METHODS

THE PILLARS OF CREATIVE COMPUTING



THE PILLARS OF CREATIVE COMPUTING



...and **lots** of data...

UNSUPERVISED LEARNING (OPTIMISATION)

Unsupervised learning is a paradigm in machine learning where, in contrast to supervised learning and semi-supervised learning, algorithms learn *patterns* exclusively from unlabeled data.

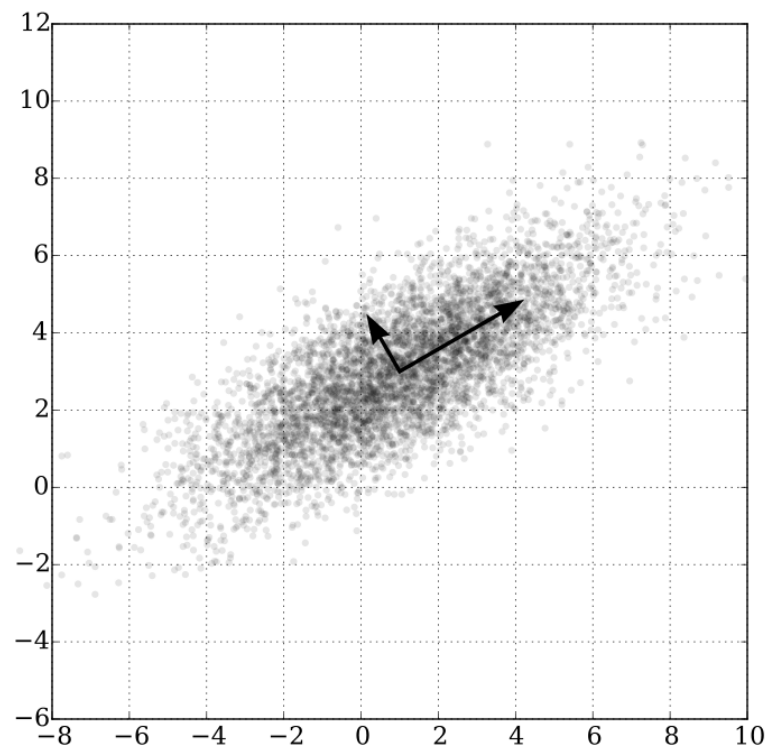
UNSUPERVISED LEARNING (OPTIMISATION)

Unsupervised learning is a paradigm in machine learning where, in contrast to supervised learning and semi-supervised learning, algorithms learn *patterns* exclusively from unlabeled data.

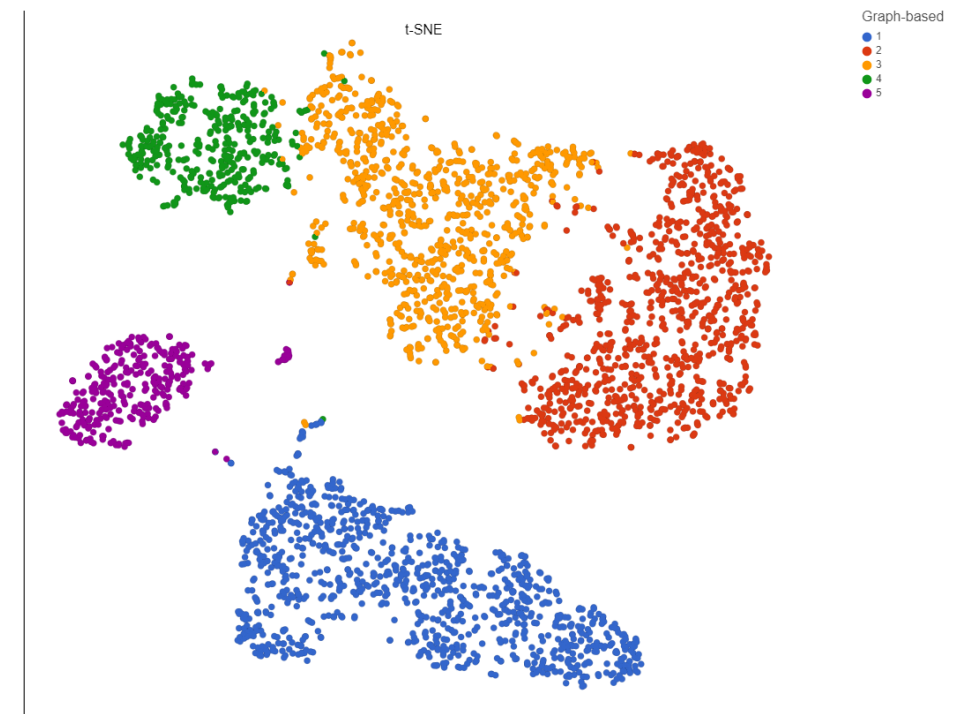
GEOMETRIC PROPERTIES OF THE FEATURE SPACE

TWO MAIN TOOLS

Two of the main methods used in unsupervised learning are **principal component** and **cluster analysis**.



Principial component analysis



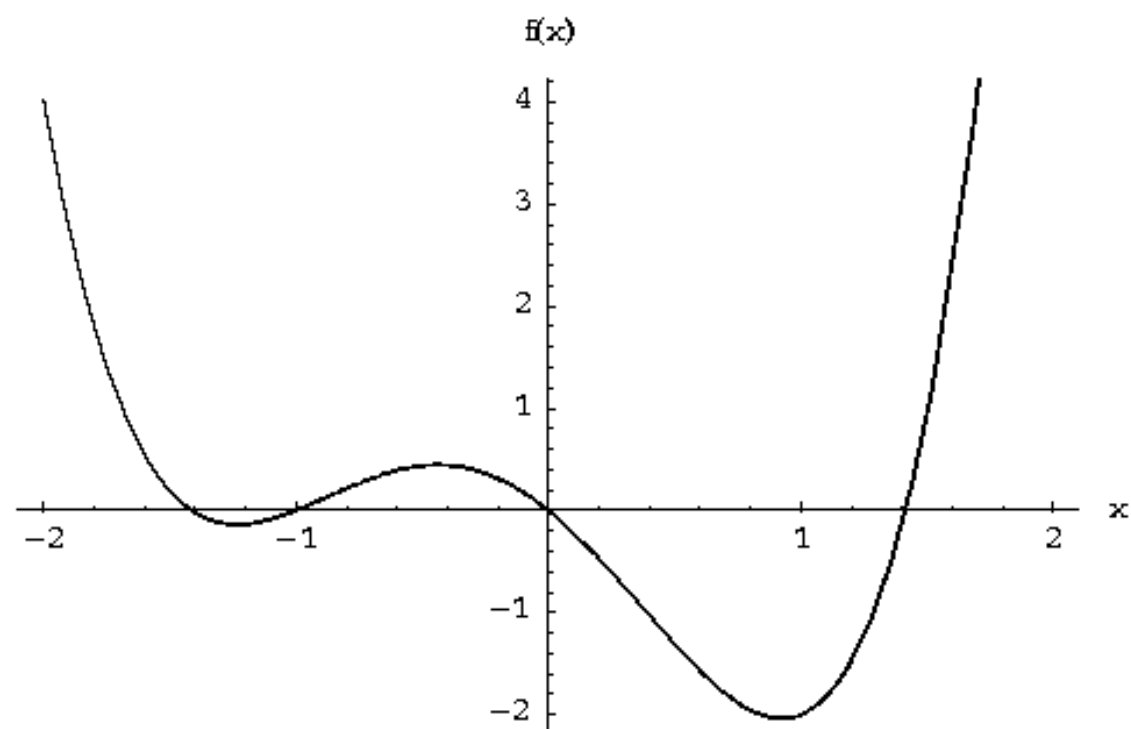
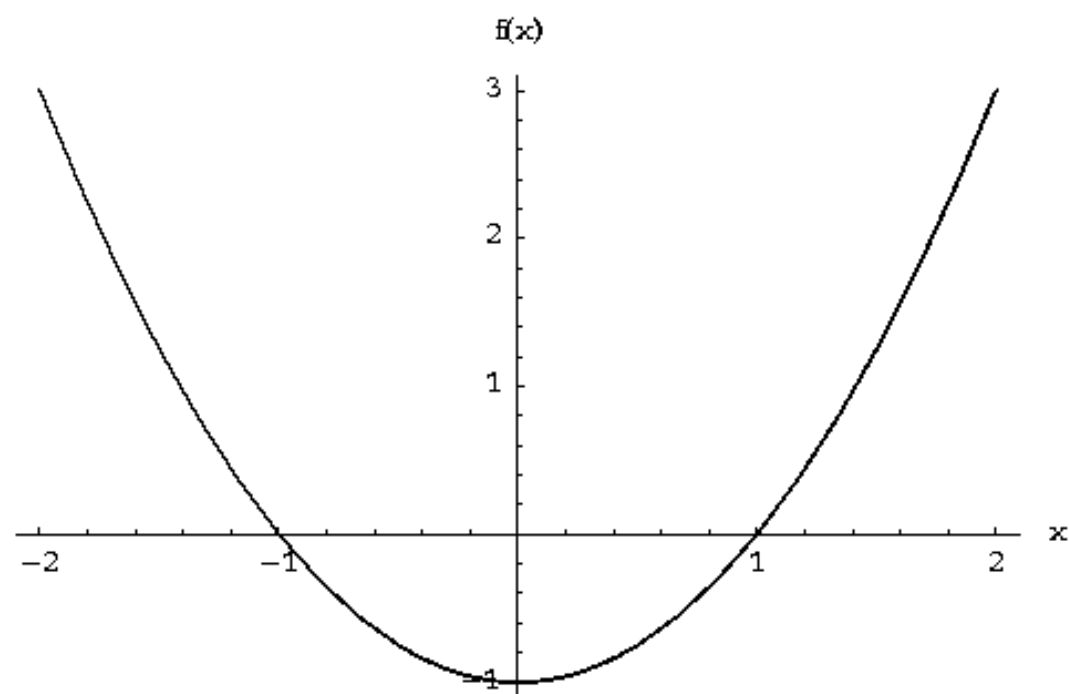
Cluster analysis

COMMON ALGORITHMS

Some of the most common algorithms used in unsupervised learning include: (1) Clustering, (2) Anomaly detection, (3) Approaches for learning latent variable models. Each approach uses several methods as follows:

- Clustering methods include: hierarchical clustering,^[9] k-means,^[10] mixture models, DBSCAN, and OPTICS algorithm
- Anomaly detection methods include: Local Outlier Factor, and Isolation Forest
- Approaches for learning latent variable models such as Expectation–maximization algorithm (EM), Method of moments, and Blind signal separation techniques (Principal component analysis, Independent component analysis, Non-negative matrix factorization, Singular value decomposition)

CONVEX VS. NON-CONVEX



CLUSTERING METHODS

SOUND-TYPES

- The *theory of sound-types* is a multi-layer framework for sound representation with multiple abstraction levels (such as CNN)

SOUND-TYPES

- The *theory of sound-types* is a multi-layer framework for sound representation with multiple abstraction levels (such as CNN)
- Sounds are described by **equivalence classes** and **probabilities**:

Equivalence classes



Timbre

Probabilities



Temporal behaviour

SOUND-TYPES

- The *theory of sound-types* is a multi-layer framework for sound representation with multiple abstraction levels (such as CNN)
- Sounds are described by **equivalence classes** and **probabilities**:

Equivalence classes



Timbre

Probabilities



Temporal behaviour

- This is realised by the **sound-types transform** (STT) and by joint probabilistic models and machine learning

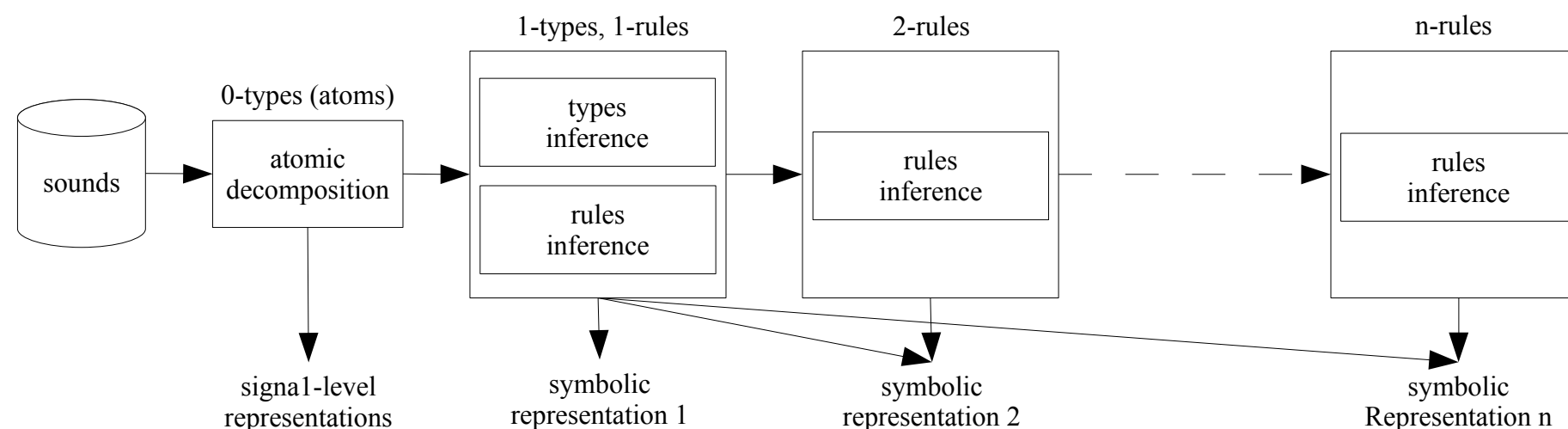
(spoiler alert: next slides will be less fun!!)

CREATION OF SOUND-TYPES 1/2

- **atomic decomposition:** subdivide a sound into small grains of approximately 40 ms called *atoms* or *0-types* overlapping in time and frequency;
- **1-types inference:** compute a set of low-level descriptors on the atoms, project the descriptors in a multi-dimensional space and compute the *clusters*; each cluster will represent a *1-type*;
- **1-rules inference:** estimate a Markov model to describe the sequences of types present in the analysed sound (*1-rules*);
- **1-level representation:** represent the sound in a symbolic language using the discovered 1-types and 1-rules;

CREATION OF SOUND-TYPES 2/2

- **n-rules inference:** estimate a Markov model of order n to describe the sequences of 1-types;
- **n-level representation:** represent the sound in a symbolic language using the discovered 1-types and n -rules;
- **repeat n-level rules and n-level representations:** until the desired level number has been reached.



SOUND-TYPES TRANSFORM 1/3

- Given a signal x of length N -samples and a window $\overset{n}{\vec{h}}$ of length n -samples, it is possible to define an **atom** as a windowed chunk of the signal of length n -samples:

$$\overset{n}{\vec{a}} = \overset{n}{\vec{h}} \cdot \overset{n}{\vec{x}} .$$

- A **sound-cluster** as a set of atoms that *lie* in a defined area of a feature space (ie. that share a *similar* set of features):

$$\overset{k_r}{\vec{c}_r} = \{ \overset{n}{\vec{a}_{r,1}}, \dots, \overset{n}{\vec{a}_{r,k_r}} \} .$$

The content of $\overset{k_r}{\vec{c}_r}$ is given by a statistical analysis applied on the feature space.

SOUND-TYPES TRANSFORM 1/3

- Given a signal x of length N -samples and a window $\overset{n}{\vec{h}}$ of length n -samples, it is possible to define an **atom** as a windowed chunk of the signal of length n -samples:

$$\overset{n}{\vec{a}} = \overset{n}{\vec{h}} \cdot \overset{n}{\vec{x}} .$$

Cut a signal into pieces...



- A **sound-cluster** as a set of atoms that *lie* in a defined area of a feature space (ie. that share a *similar* set of features):

$$\overset{k_r}{\vec{c}_r} = \{ \overset{n}{\vec{a}}_{r,1}, \dots, \overset{n}{\vec{a}}_{r,k_r} \} .$$

The content of $\overset{k_r}{\vec{c}_r}$ is given by a statistical analysis applied on the feature space.

SOUND-TYPES TRANSFORM 1/3

- Given a signal x of length N -samples and a window $\overset{n}{\vec{h}}$ of length n -samples, it is possible to define an **atom** as a windowed chunk of the signal of length n -samples:

$$\overset{n}{\vec{a}} = \overset{n}{\vec{h}} \cdot \overset{n}{\vec{x}} .$$

Cut a signal into pieces...

- A **sound-cluster** as a set of atoms that *lie* in a defined area of a feature space (ie. that share a *similar* set of features):

...then group pieces that

are *similar*

$$\overset{k_r}{\vec{c}_r} = \{ \overset{n}{\vec{a}_{r,1}}, \dots, \overset{n}{\vec{a}_{r,k_r}} \} .$$

The content of $\overset{k_r}{\vec{c}_r}$ is given by a statistical analysis applied on the feature space.

SOUND-TYPES TRANSFORM 2/3

- A sound-cluster has an associate **sound-type** $\vec{\tau}_r^n$, defined as the weighted sum of all the atoms in the sound-cluster where the weights $\vec{\omega}_r^{k_r}$ are the distances of each atom to the center of the cluster:

$$\vec{\tau}_r^n = \sum_{j=1}^{k_r} \vec{a}_{r,j}^n \cdot \omega_{r,j}$$

with $\omega_{r,j} \in \vec{\omega}_r^{k_r}$.

- The set of sound-types in the signal $\vec{x}^N$ is called **dictionary**:

$$\mathcal{D}_{\vec{x}^N} = \{\vec{\tau}_1^n, \dots, \vec{\tau}_r^n\}.$$

SOUND-TYPES TRANSFORM 2/3

- A sound-cluster has an associate **sound-type** $\vec{\tau}_r^n$, defined as the weighted sum of all the atoms in the sound-cluster where the weights $\vec{\omega}_r^{k_r}$ are the distances of each atom to the center of the cluster:

$$\vec{\tau}_r^n = \sum_{j=1}^{k_r} \vec{a}_{r,j}^n \cdot \omega_{r,j}$$

Create an intermediate
representation of each *type* of
sound discovered



with $\omega_{r,j} \in \vec{\omega}_r^{k_r}$.

- The set of sound-types in the signal $\vec{x}^N$ is called **dictionary**:

$$\mathcal{D}_{\vec{x}^N} = \{\vec{\tau}_1^n, \dots, \vec{\tau}_r^n\}.$$

SOUND-TYPES TRANSFORM 3/3

- Finally, it is possible to define the *sound-types transform* as a function of time and frequency obtained by multiplying the sound-types in a given dictionary with complex sinusoids:

$$\vec{\Phi}_{\vec{k}}^N = \sum_{i=0}^{N/t} \vec{\tau}_{r,p}^n \cdot e^{-j \cdot \frac{2 \cdot \pi}{n} \cdot \vec{k}}$$

where $\vec{k}^n = \{f_1, \dots, f_n\}$ is a vector of frequencies.

SOUND-TYPES TRANSFORM 3/3

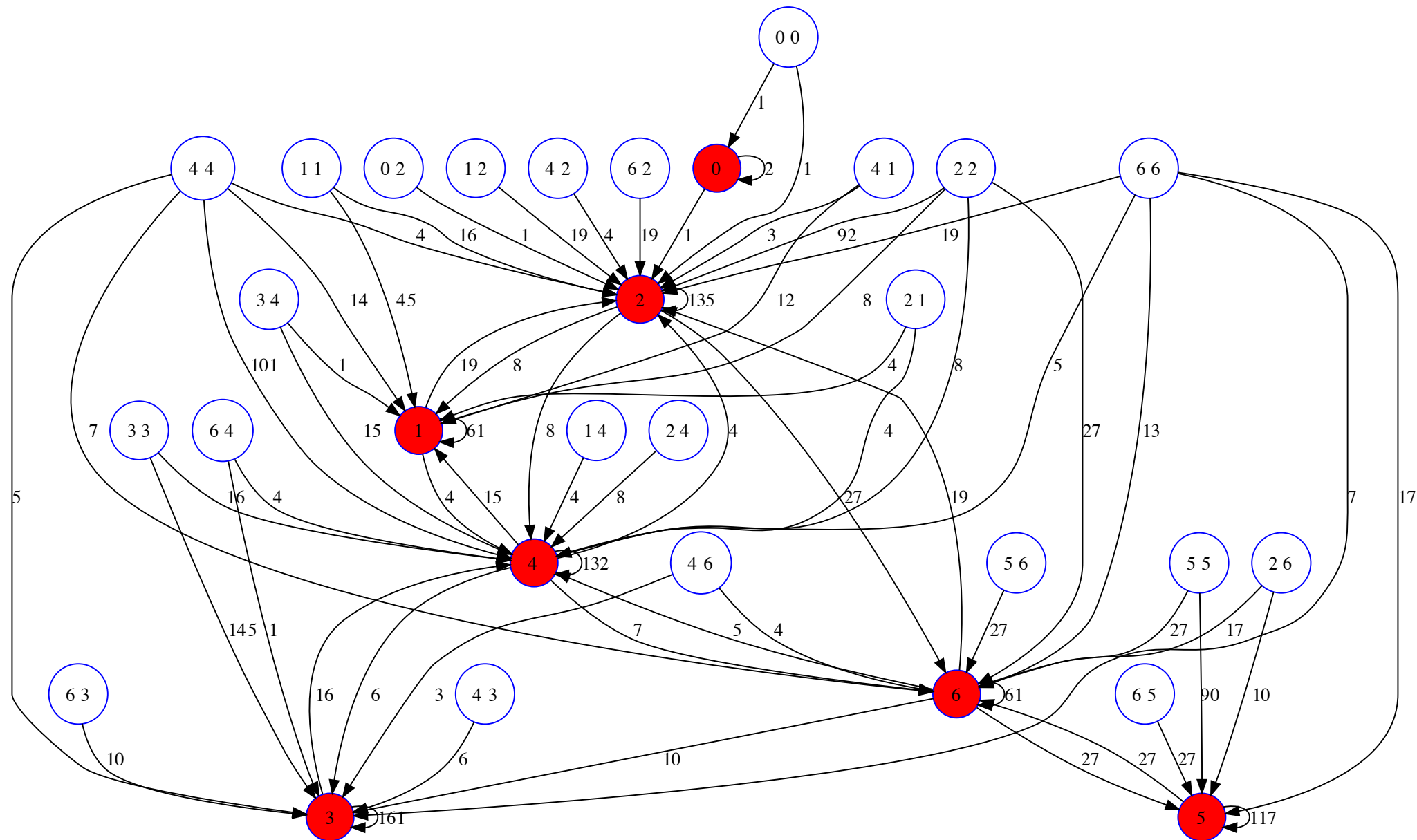
- Finally, it is possible to define the *sound-types transform* as a function of time and frequency obtained by multiplying the sound-types in a given dictionary with complex sinusoids:

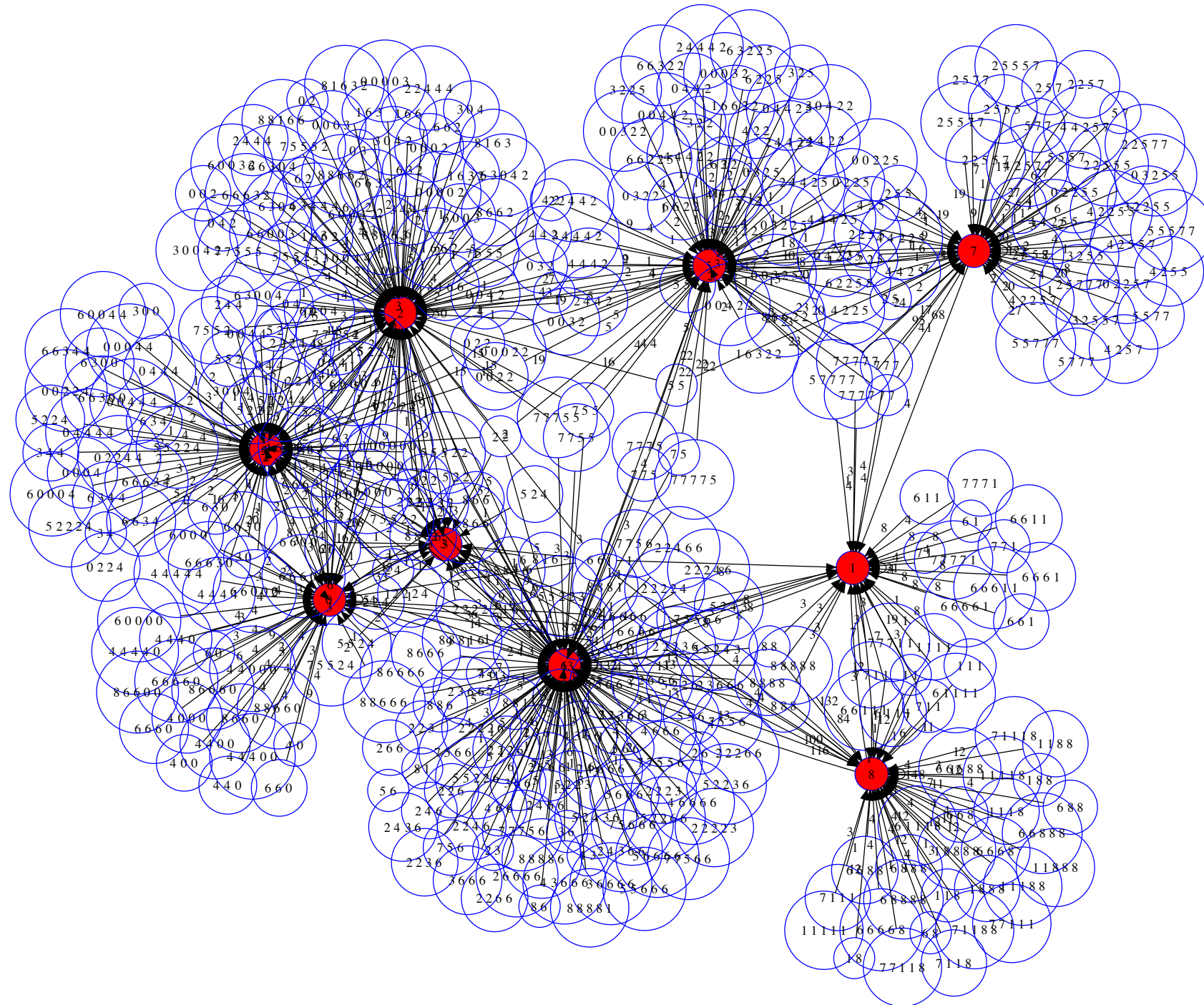
$$\vec{\Phi}_{\vec{k}}^N = \sum_{i=0}^{N/t} \vec{\tau}_{r,p}^n \cdot e^{-j \cdot \frac{2 \cdot \pi}{n} \cdot \vec{k}}$$

Create the final projective representation at the appropriate abstraction level

where $\vec{k}^n = \{f_1, \dots, f_n\}$ is a vector of frequencies.

SOUND-TYPES AT ABSTRACTION LEVEL 2





EXAMPLES

Probabilistic generation / sound-types matching

- Bass sample and its probabilistic generation.
- A probabilistic generation of an orchestral sample.
- Hybridization between instruments and voices (Gervasoni).
- Hybridization between Cage and Beach boys.
- Hybridization between Cage and Lachenmann's piece.

EXAMPLES

Probabilistic generation / sound-types matching

- Bass sample and its probabilistic generation.
- A probabilistic generation of an orchestral sample.
- Hybridization between instruments and voices (Gervasoni).
- Hybridization between Cage and Beach boys.
- Hybridization between Cage and Lachenmann's piece.

We will experiment with sound-types during our next lab!

PRINCIPAL COMPONENT METHODS

MATCHING PURSUIT (1)

Matching pursuit (MP) is a **sparse approximation** algorithm which finds the "best matching" projections of multidimensional data onto the span of an over-complete (i.e., redundant) dictionary D

$$f(t) \approx \hat{f}_N(t) := \sum_{n=1}^N a_n g_{\gamma_n}(t)$$

where g_{γ_n} is the γ_n th column of the matrix D and a_n is the scalar weighting

$$\min_x \|f - Dx\|_2^2 \quad \text{subject to} \quad \|x\|_0 \leq N,$$

MATCHING PURSUIT (2)

- The algorithm converges (i.e.

$$R_n \rightarrow 0$$

) for any f that is in the space spanned by the dictionary.

- The error

$$\|R_n\|$$

decreases monotonically.

- As at each step, the residual is orthogonal to the selected filter, the energy conservation equation is satisfied for each N :

$$\|f\|^2 = \|R_{N+1}\|^2 + \sum_{n=1}^N |a_n|^2$$

.

- In the case that the vectors in D are orthonormal, rather than being redundant, then MP is a form of **principal component analysis**

MA.P.L.E.

Matching pursuit linear expansion

<https://github.com/CarmineCella/maple>

Maple is an experimental framework for sound processing based on time/frequency decomposition and dictionary learning.

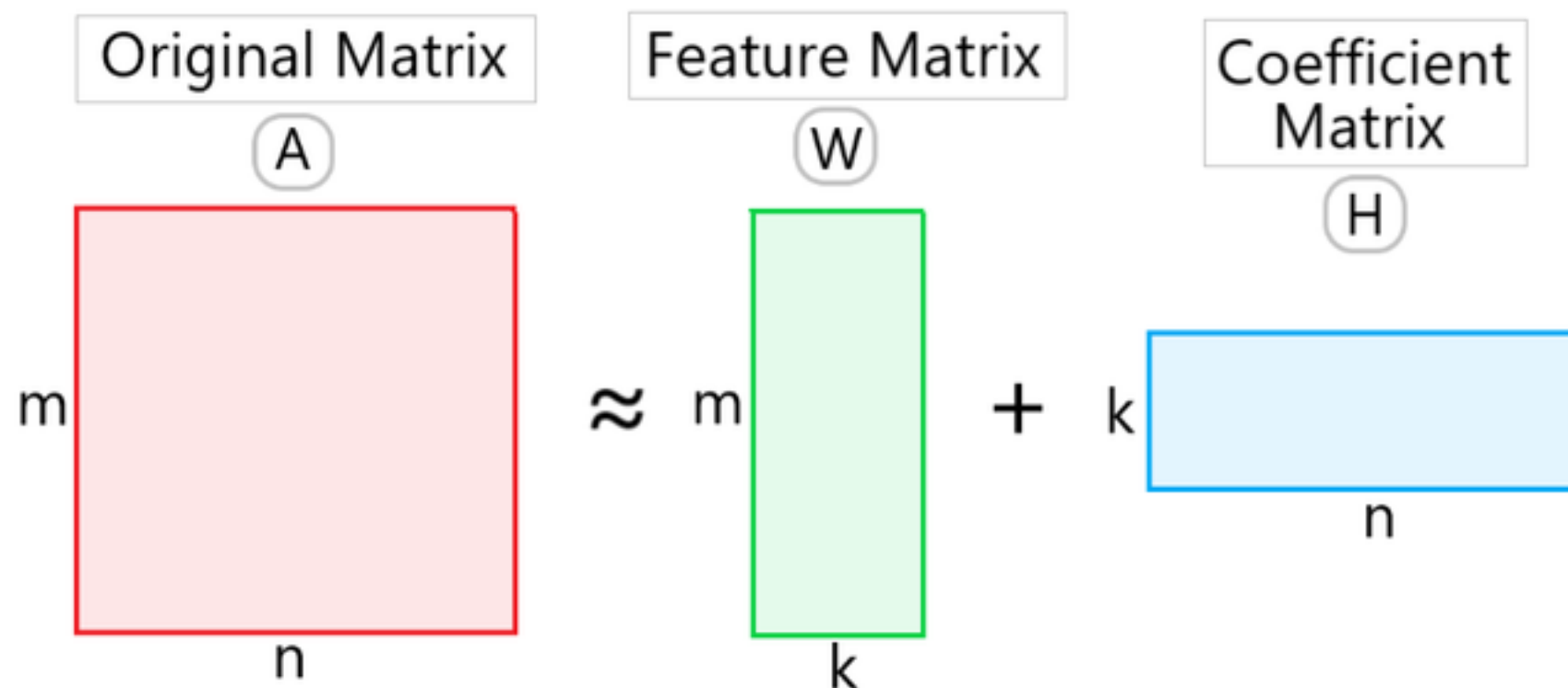
An interesting effect that Maple can do is sound hybridisation.

- [Vox and clarinet]
- [Vox and door noises]
- [Vox and kitchen noises (onsets)]
 - [Vox and orchestra (onsets)]
- [Vox and various instruments (onsets)]
 - [Vox and animals (onsets)]

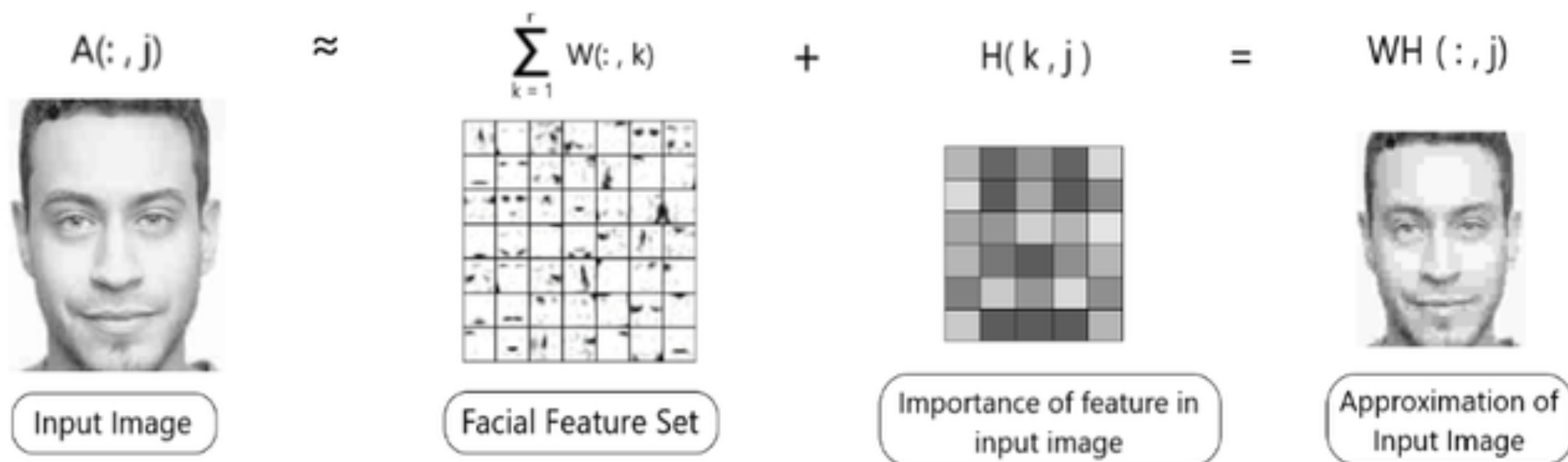
NON-NEGATIVE MATRIX FACTORIZATION (1)

NMF approximates a matrix $\mathbf{X}$ with a low-rank matrix approximation such that $\mathbf{X} \approx \mathbf{WH}$.

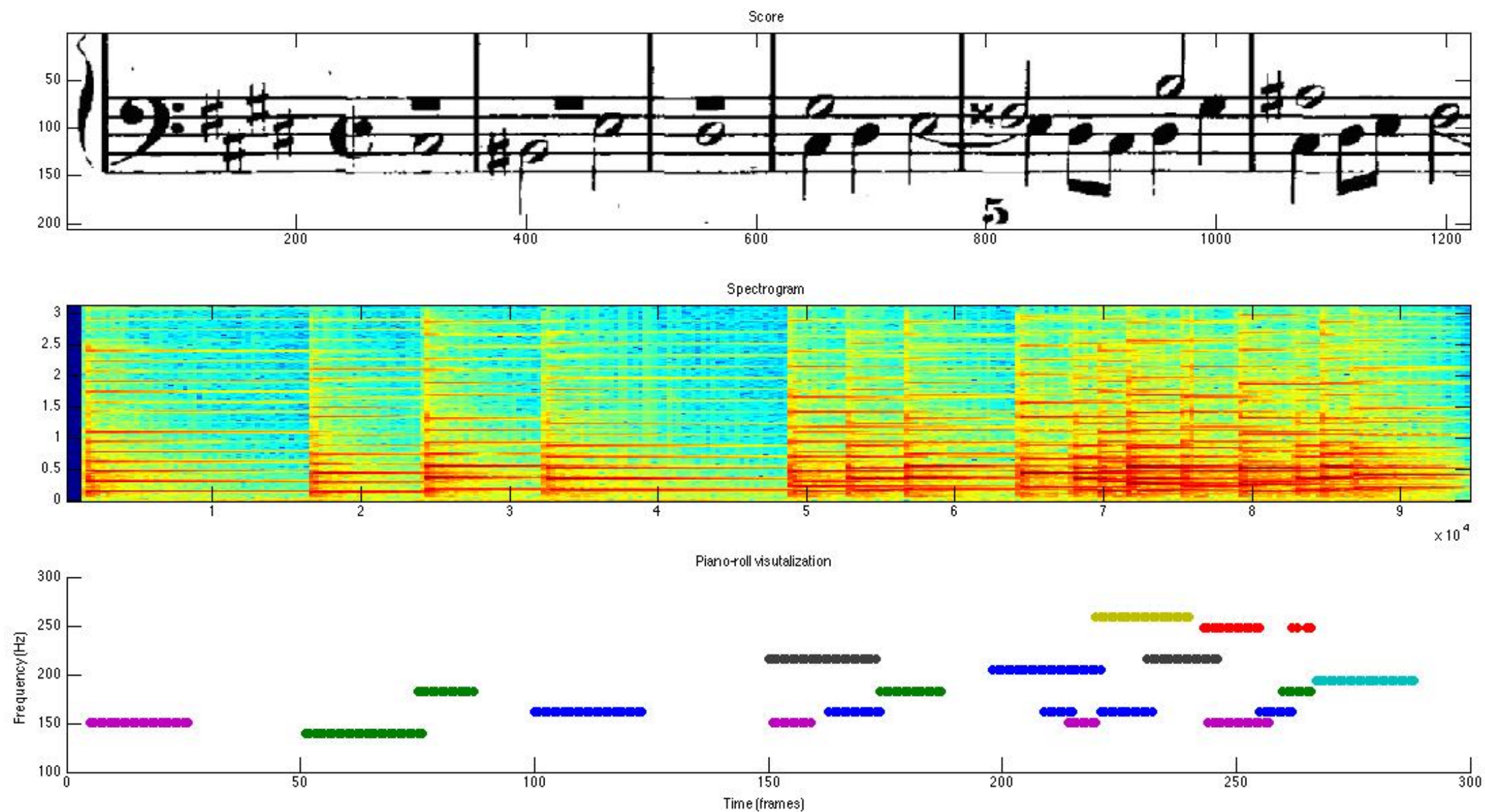
For the discussion in this paper, we'll assume that $\mathbf{X}$ is set up so that there are n data points each with p dimensions, and every *column* of $\mathbf{X}$ is a data point, i.e. $\mathbf{X} \in \mathbb{R}^{p \times n}$.



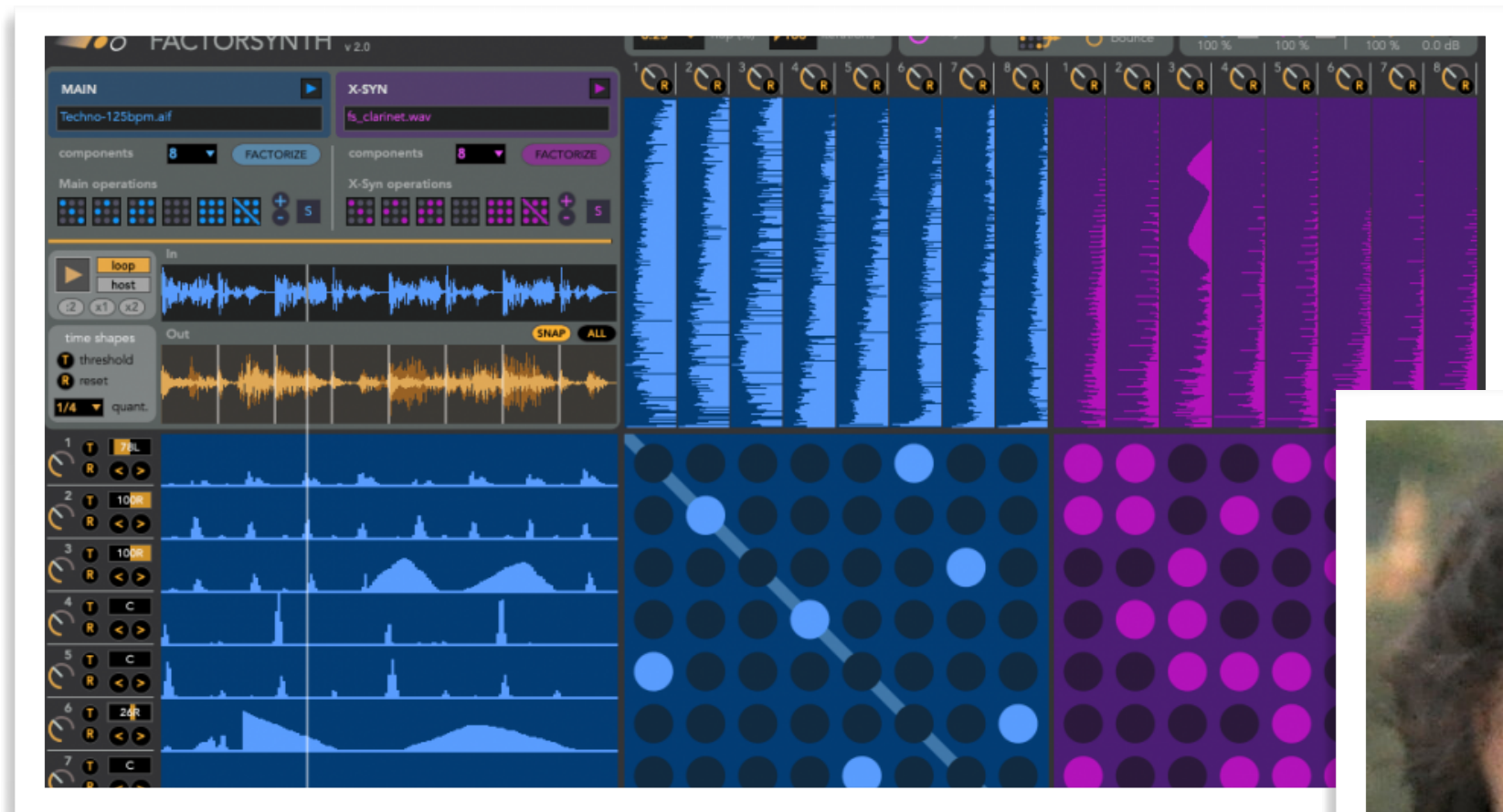
NNMF EXAMPLE: IMAGE APPROXIMATION



NNMF EXAMPLE: MUSIC TRANSCRIPTION



NNMF EXAMPLE: FACTOR SYNTH (JJ BURRED)



EVOLUTIONARY APPROACHES

A BIOLOGY LESSON

Every organism has a set of rules describing how that organism is built up from the tiny building blocks of life.

These rules are encoded in the genes of an organism, which in turn are connected together into long strings called **chromosomes**.

A BIOLOGY LESSON

Every organism has a set of rules describing how that organism is built up from the tiny building blocks of life.

These rules are encoded in the genes of an organism, which in turn are connected together into long strings called **chromosomes**.

Genetic Algorithms are a way of solving problems by mimicking the same processes mother nature uses.

They use a combination of **selection**, **recombination** and **mutation** to evolve a solution to a problem.

BASIC ALGORITHM

At the beginning of a run of a genetic algorithm a large population of *random* chromosomes is created. Each one, will represent a different solution to the problem.

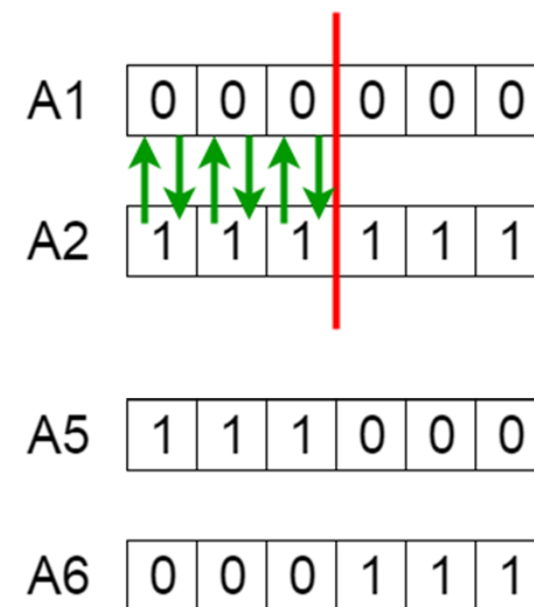
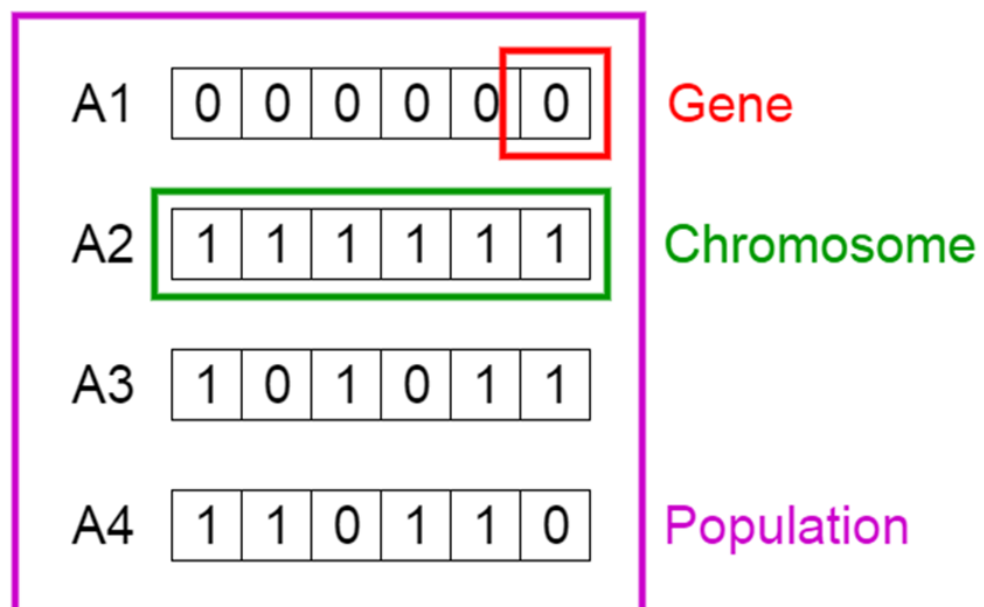
Then, the following steps are repeated until a solution is found:

1. Test each chromosome to see how good it is at solving the problem and assign a **fitness** score accordingly.
2. Select **two** members from the current population. The chance of being selected is proportional to the chromosomes fitness. *Roulette wheel* selection is a commonly used method.
3. Dependent on the **crossover** rate crossover the bits from each chosen chromosome at a randomly chosen point.
4. Step through the chosen chromosomes bits and flip dependent on the **mutation** rate.
5. Repeat step 2, 3, 4 until a new population of N members has been created.

IMPLEMENTATION OVERVIEW

Pseudocode

```
START
Generate the initial population
Compute fitness
REPEAT
    Selection
    Crossover
    Mutation
    Compute fitness
UNTIL population has converged
STOP
```



CROSSOVER RATE

This is simply the chance that two chromosomes will swap their bits. A good value for this is around 0.7. Crossover is performed by selecting a random gene along the length of the chromosomes and swapping all the genes after that point.

e.g. Given two chromosomes

10001001110010010
01010001001000011

Choose a random bit along the length, say at position 9, and swap all the bits after that point

so the above become:

10001001101000011
01010001010010010

MUTATION RATE

This is the chance that a bit within a chromosome will be flipped (0 becomes 1, 1 becomes 0). This is usually a very low value for binary encoded genes, say 0.001

So whenever chromosomes are chosen from the population the algorithm first checks to see if crossover should be applied and then the algorithm iterates down the length of each chromosome mutating the bits if applicable.

A SIMPLE PROBLEM

Given the digits 0 through 9 and the operators +, -, * and /, find a sequence that will represent a given target number. The operators will be applied sequentially from left to right as you read.

Given the target number 23, the sequence $6+5*4/2+1$ would be one possible solution.

If 75.5 is the chosen number then $5/2+9*7-5$ would be a possible solution.

ENCODING STAGE

Four bits are required to represent the range of characters used:

0:	0000
1:	0001
2:	0010
3:	0011
4:	0100
5:	0101
6:	0110
7:	0111
8:	1000
9:	1001
+	1010
-	1011
*	1100
/	1101

So now you can see that the solution mentioned above for 23, '6+5*4/2+1' would be represented by nine genes like so:

0110 1010 0101 1100 0100 1101 0010 1010 0001

6 + 5 * 4 / 2 + 1

FITNESS FUNCTION

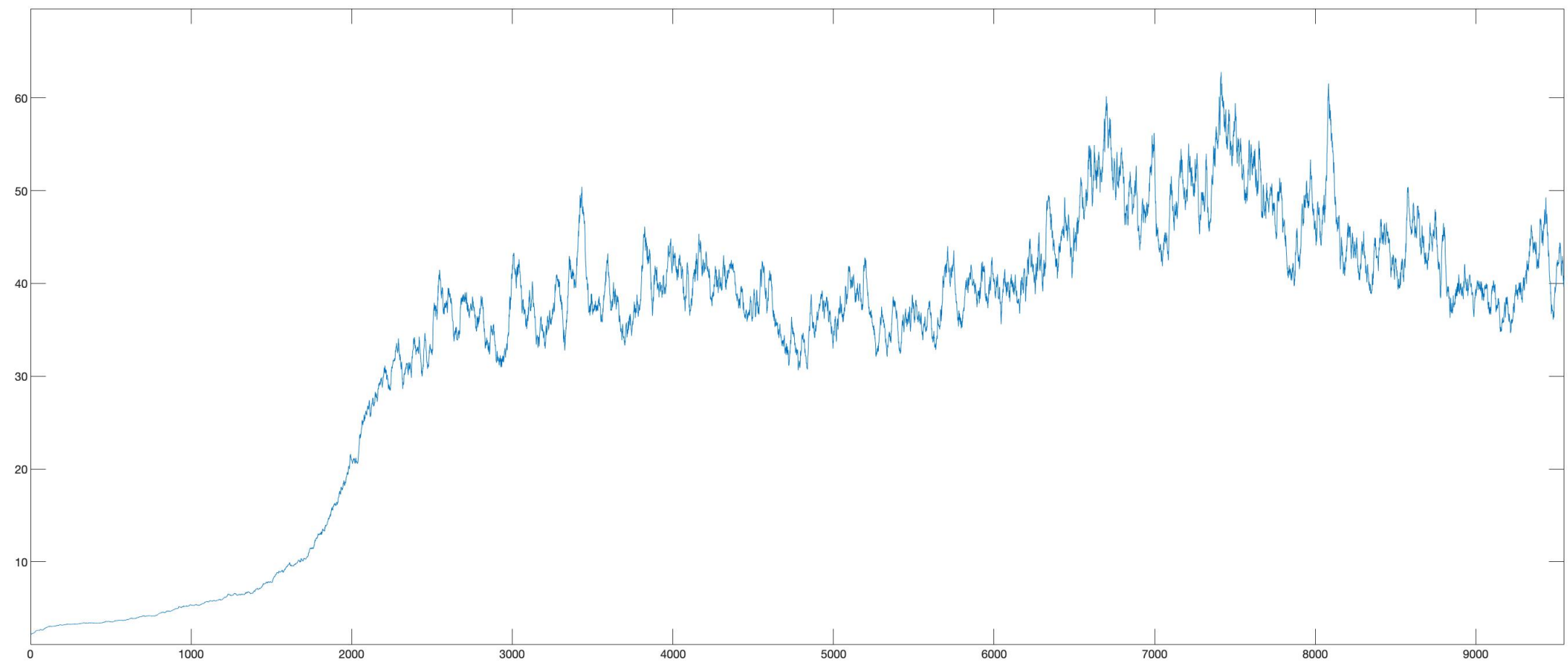
This can be the most difficult part of the algorithm to figure out. It really depends on what problem you are trying to solve but the general idea is to give a higher fitness score the closer a chromosome comes to solving the problem. A fitness score can be assigned that's inversely proportional to the difference between the solution and the value a decoded chromosome represents.

If we assume the target number for the remainder of the tutorial is 42, the chromosome mentioned above:

011010100101110001001101001010100001

has a fitness score of $1/(42-23)$ or $1/19$.

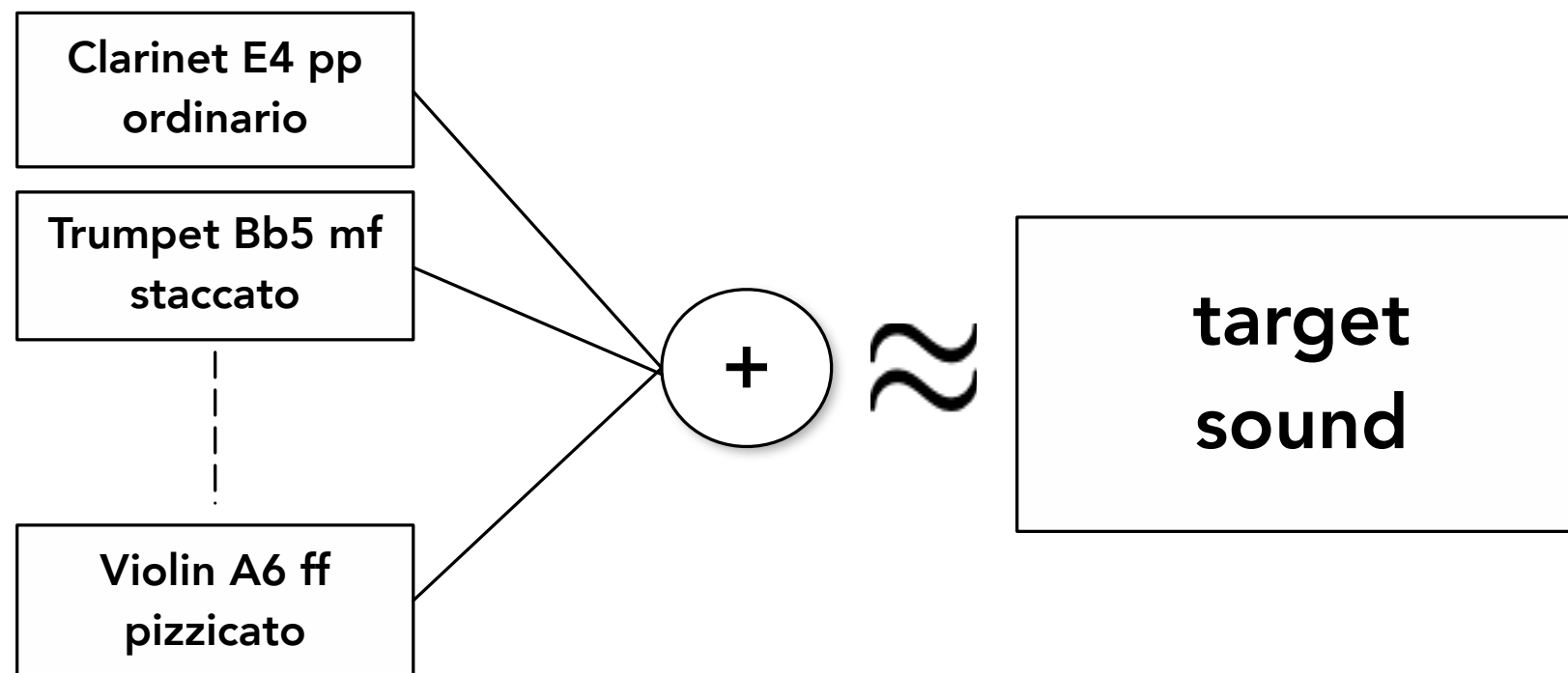
EXAMPLE OF FITNESS



COMPUTER-ASSISTED ORCHESTRATION

A PRELIMINARY DEFINITION

The process of searching for the best combinations of orchestral sounds to match a target sound under specified metric and constraints.



ORCHESTRATION: PROJECTION

7 Muovendo ♩ = 78

103 **8**

Picc. *ff*

Fl. *fff*

Ob. *ff* *fff* *simile*

Eng. horn *ff* *fff* *simile*

Cl. *fff*

B. Cl. *fff*

Drum *fff*

This image shows a page of an orchestral score for a piece titled 'Muovendo' with a tempo of 78 beats per minute. The page is numbered 103 and features a large section number '8'. The score is for a woodwind and percussion ensemble. The instruments listed on the left are Piccolo (Picc.), Flute (Fl.), Oboe (Ob.), English Horn (Eng. horn), Clarinet (Cl.), Bass Clarinet (B. Cl.), and Drum. The Piccolo part begins with a forte (ff) dynamic. The Flute part starts with a fortissimo (fff) dynamic and includes a trill marked 'tr'. The Oboe and English Horn parts have dynamics ranging from ff to fff, with some passages marked 'simile'. The Clarinet and Bass Clarinet parts are marked fff. The Drum part is also marked fff. The score includes various musical notations such as slurs, ties, and articulation marks. The key signature has one sharp (F#), and the time signature is 4/4.

ORCHESTRATION: PROJECTION

7 Muovendo ♩ = 78

103 **8**

Picc. *ff*

Fl. *fff*

Ob. *ff* *sfff* *simile*

Eng. horn *ff* *sfff* *simile*

Cl. *fff*

B. Cl. *fff*

Drum *fff*

The image displays a page from an orchestral score, specifically measures 103 and 104. The tempo is marked 'Muovendo' with a quarter note equal to 78 beats per minute. The key signature has one sharp (F#). The score includes parts for Piccolo (Picc.), Flute (Fl.), Oboe (Ob.), English Horn (Eng. horn), Clarinet (Cl.), Bass Clarinet (B. Cl.), and Drum (Drum). Dynamics are indicated by 'ff' (fortissimo) and 'fff' (fortississimo). Performance instructions like 'tr' (trill), 'simile' (simile), and 'dim.' (diminuendo) are present. Measure numbers 103 and 104 are clearly visible at the beginning of the staves.

PROJECTION

Signal space
(timbre)

ORCHESTRATION: INDUCTION?

Signal space
(timbre)

ORCHESTRATION: INDUCTION?

Fl *ff*

Ob *pp*

Cl *pp*

Bn *mf*

Hn *ff*

Tp *mf* Straight

Tbn *pp*

Vn *mf* *mf* pont | 4c

Va *ff* nonvib | 3c

Vc *ff* nonvib | 2c

Cb *ff* nonvib | 4c



Signal space
(timbre)

PROBLEM STATEMENT

- How an orchestra can be used to reproduce a **target timbre** within a compositional context?

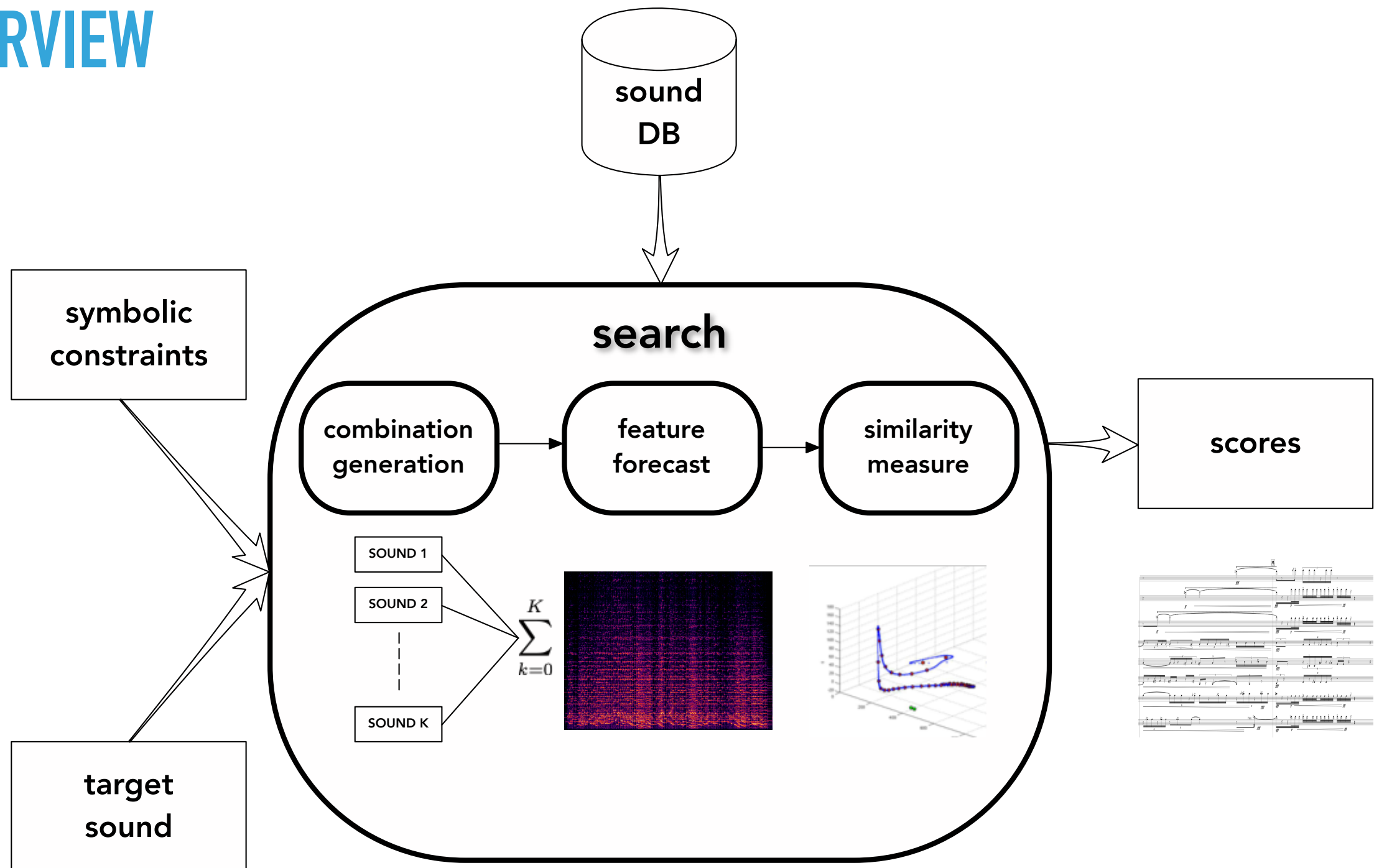
PROBLEM STATEMENT

- How an orchestra can be used to reproduce a **target timbre** within a compositional context?
- How can we find a combination of instruments that:
 - ◆ Best matches a given target sound?
 - ◆ Fits writing constraints specified by the composer?

PROBLEM STATEMENT

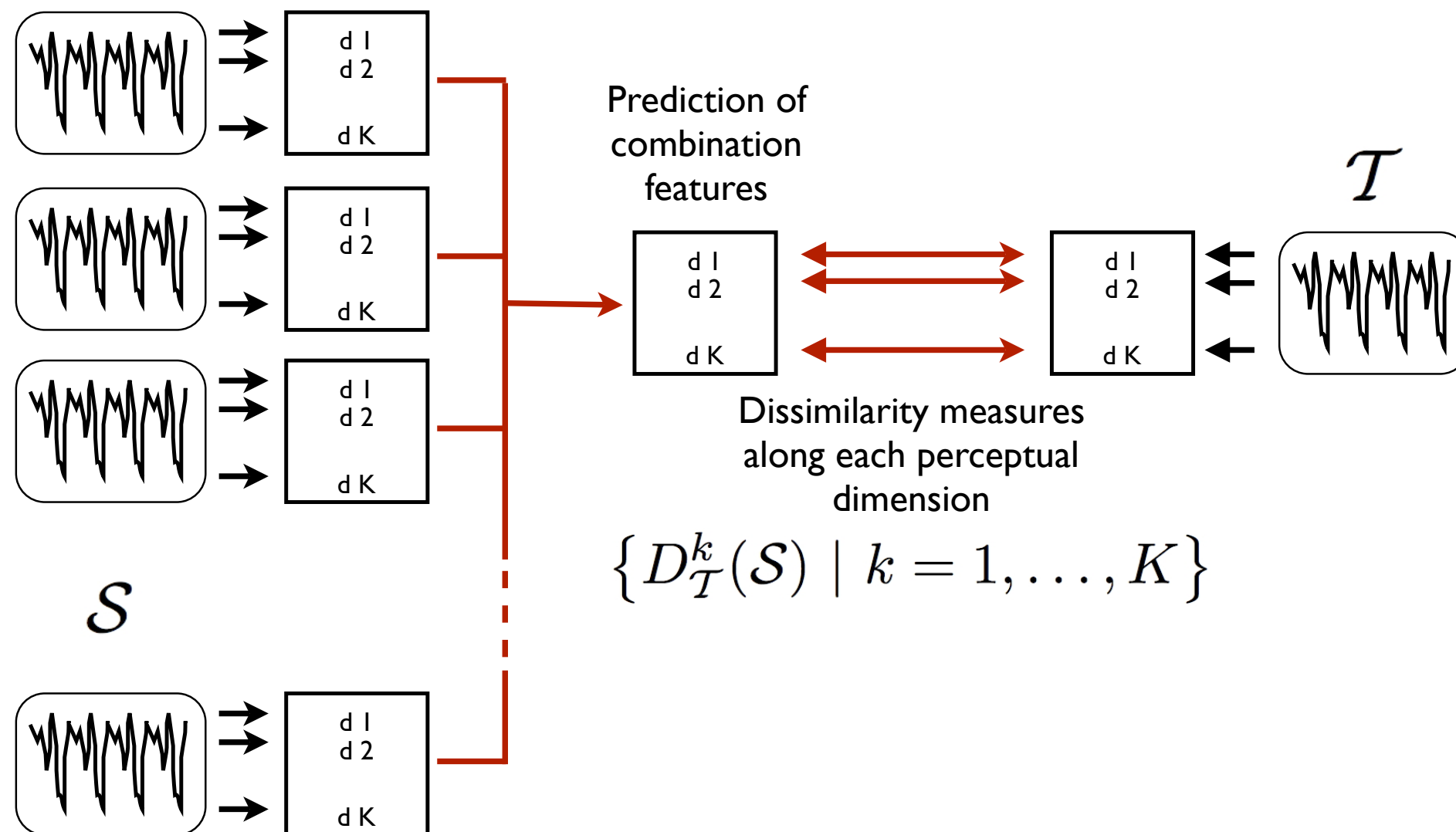
- How an orchestra can be used to reproduce a **target timbre** within a compositional context?
- How can we find a combination of instruments that:
 - ◆ Best matches a given target sound?
 - ◆ Fits writing constraints specified by the composer?
- More formally:
 - ◆ A combinatorial optimisation problem defined on timbre description
 - ◆ A constraint solving problem on the variables of musical writing

OVERVIEW



FEATURES FORECAST AND DISSIMILARITY

The target sound and the DB are embedded in the same feature space; then a forecast for each combination is created and compared to the target



ORCHIS/ORCHIDÉE & ORCHIDS

The screenshot shows the ORCHIS/ORCHIDÉE software interface. At the top, there is a timeline bar with a play button and a time display of 5.007 sec. Below this, the interface is divided into several sections:

- Settings:** Includes checkboxes for "Harmonic filtering" (checked) and "Disregard onsets" (checked). It also shows "Search depth: 100", "Max solutions: 100", and "Algorithm: Genetic".
- Overview:** A scatter plot titled "SpectralSpreadM..." with axes ranging from 0.50 to 1.00. It shows a distribution of points with a blue circle highlighting a specific point.
- Advanced Research Parameters:** A button labeled "Set Current Solution As Target".
- Solution 3/100:** A musical score for five instruments: Flute (Fl), Clarinet in Bb (ClBb), Horn (Hn), Violin (Va), and Violoncello (Vc). The score shows notes with dynamic markings and durations, such as "mf lasting_half_sec" for the Flute and "ff ord" for the Violin.

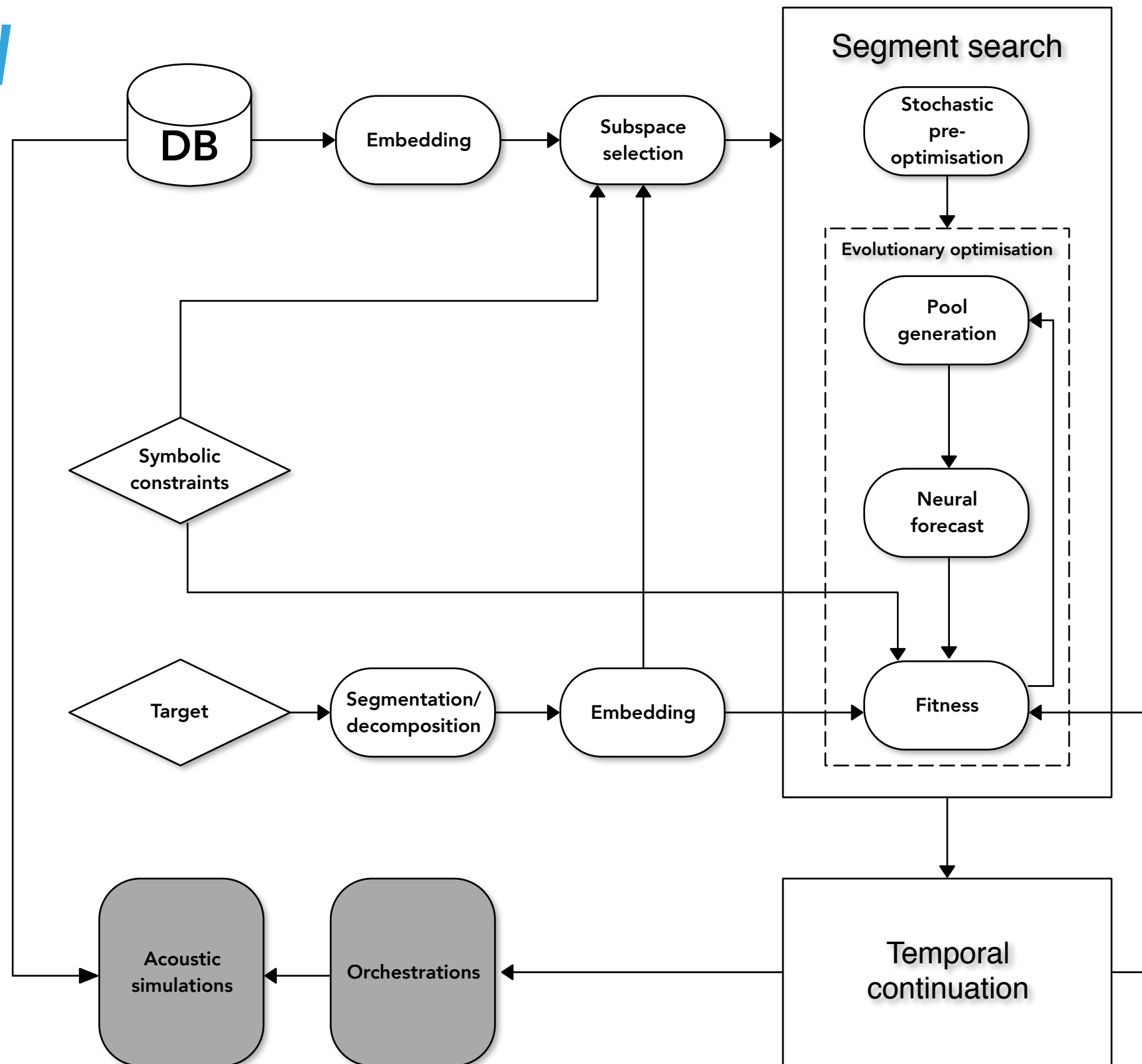
The screenshot shows the Orchis 0.62 Automatic Orchestration Tool interface. At the top, there is a logo of a green flower in a brown pot. The title "Orchis 0.62" is prominently displayed, followed by "Automatic Orchestration Tool" and the copyright notice "Grégoire Carpentier - © IRCAM 2010 HEM-GENEVE-2010_12". There are "Help" and "Abort" buttons in the top right corner.

The interface is divided into several sections:

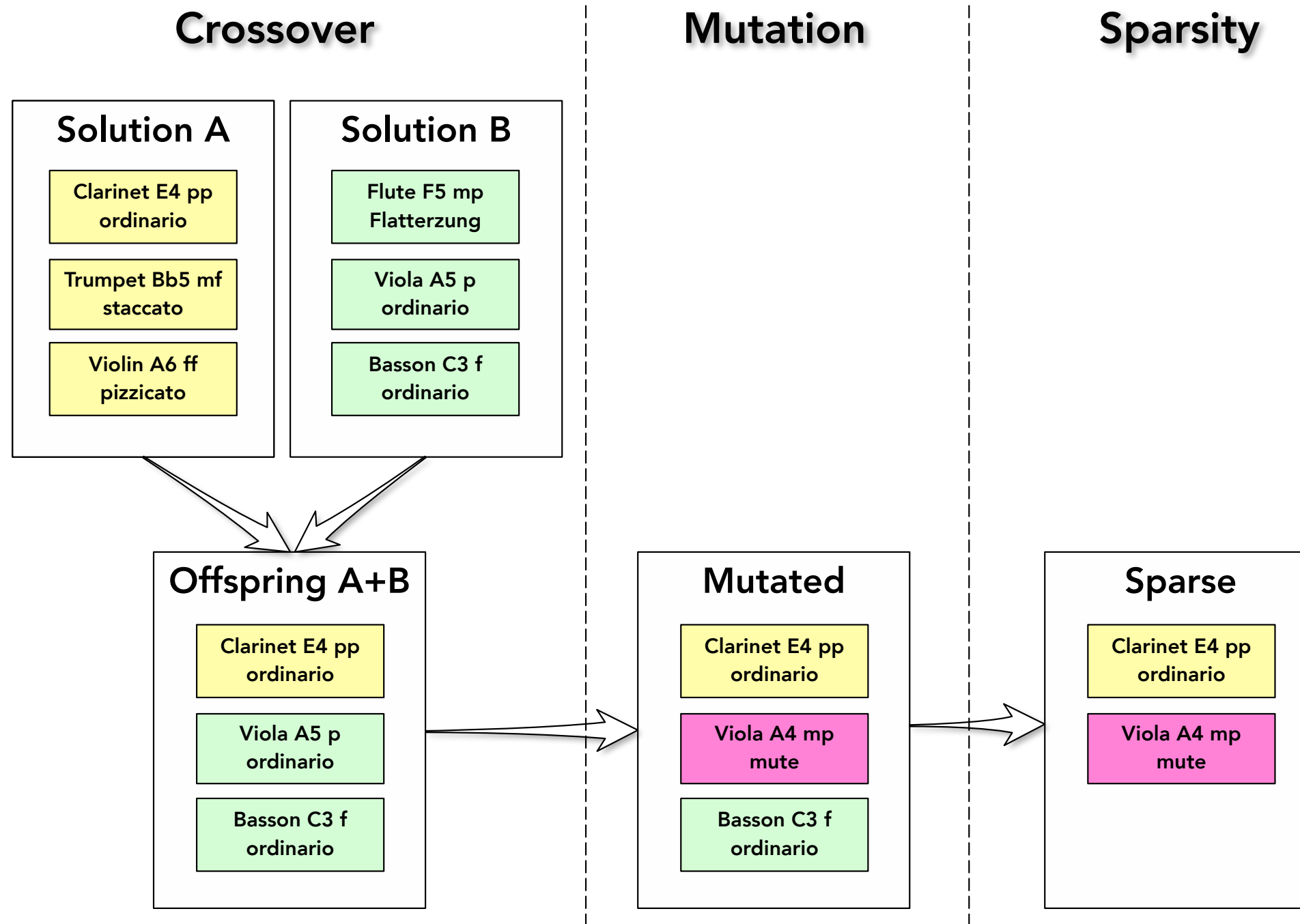
- Server:** A section with a "SERVER" dropdown menu and a text input field containing "/ready".
- SETTINGS, DATABASE, SESSION:** Three buttons for navigating between different settings.
- Target:** A section with a "Static" dropdown menu, an "Open" button, and a text input field containing "Beethoven_chord2.wav". There is also a "More" button and a red stop button.
- Waveform:** A large blue area displaying a waveform of the target audio file. The waveform is divided into three sections: a purple section from 0.00 to 1000.00, a blue section, and a teal section. A vertical yellow bar is on the right side of the waveform.

ORCHIDEA

OVERVIEW



EVOLUTIONARY OPTIMISATION

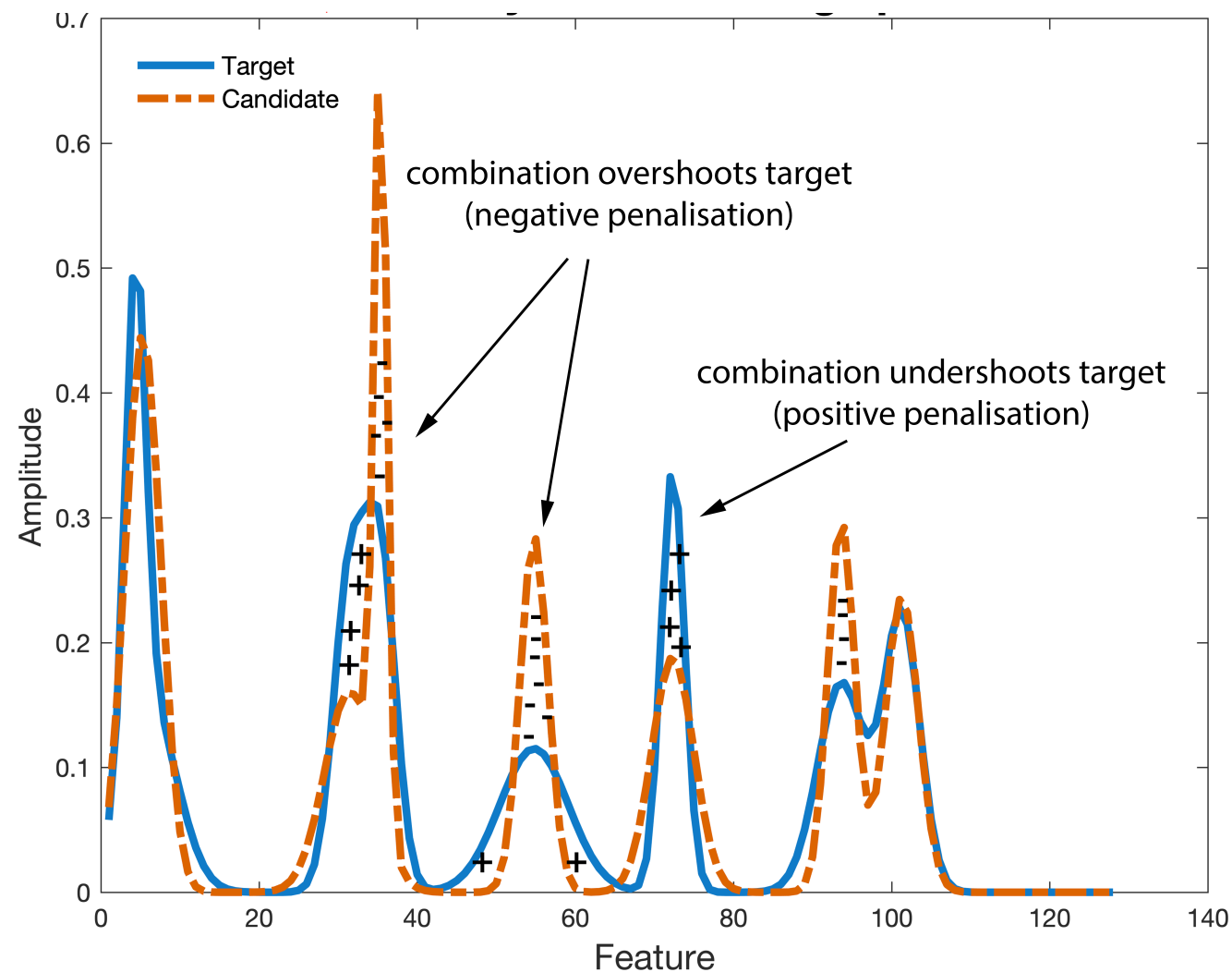


SPECTRAL COST

$$\mathcal{D}(x^t, \tilde{x}^t) = \lambda_1 \sum_k \delta_k^t (x_k^t - \tilde{x}_k^t) + \lambda_2 \sum_k \bar{\delta}_k^t |x_k^t - \tilde{x}_k^t| \quad (22)$$

where $x^t, \tilde{x}^t$ are the feature vectors of sub-target and of the candidate combination at time t respectively, k is the feature number, λ_1, λ_2 are weighting factors and

$$\delta_k = \begin{cases} 1, & \text{if } x_k \geq \tilde{x}_k \\ 0, & \text{otherwise} \end{cases}, \quad \bar{\delta}_k = \begin{cases} 1, & \text{if } x_k < \tilde{x}_k \\ 0, & \text{otherwise} \end{cases}.$$



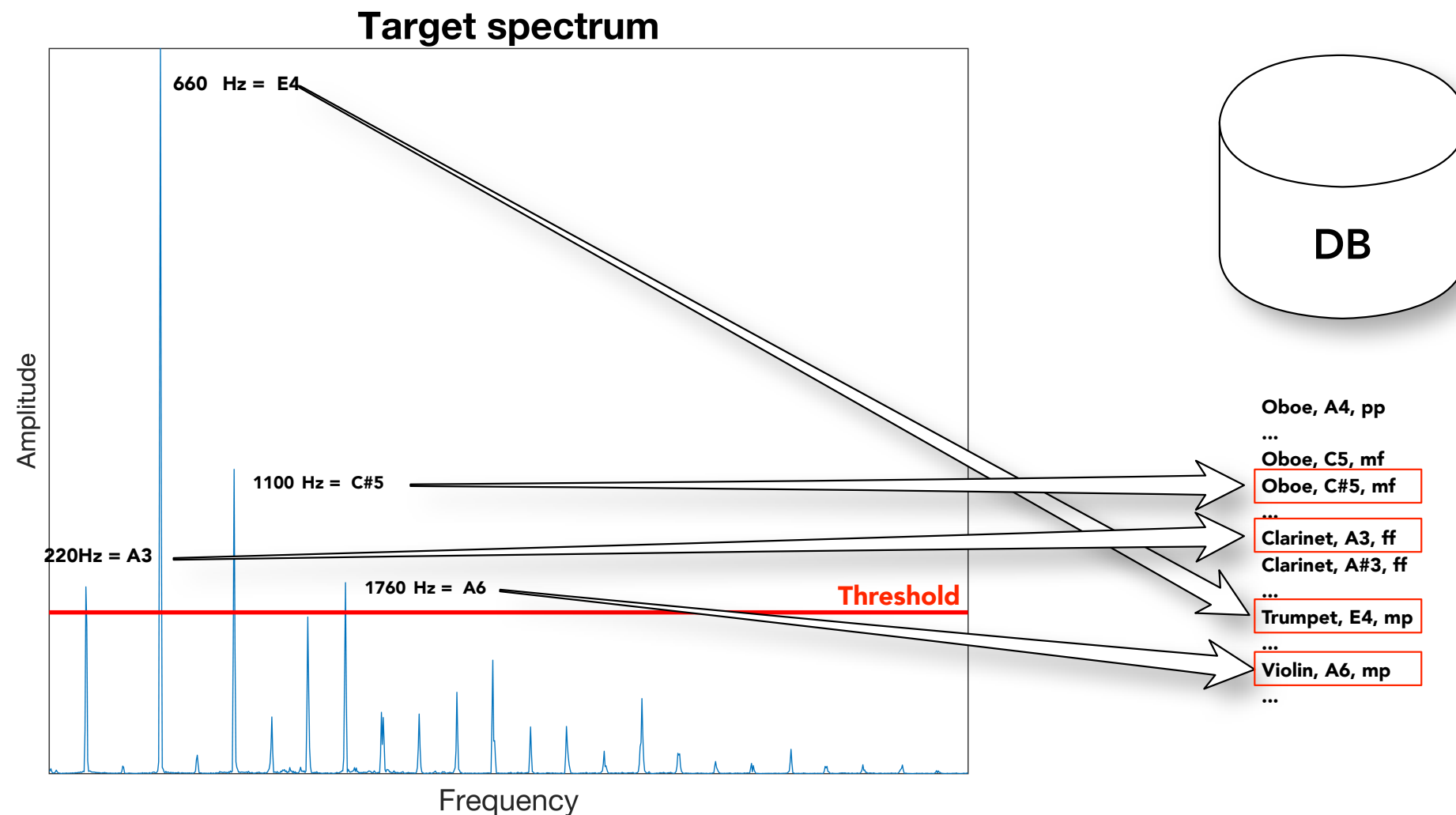
SYMBOLIC CONSTRAINTS

$$\mathcal{S}(C, \tilde{x}^t) = \sum_l \phi_l^t \quad (23)$$

where

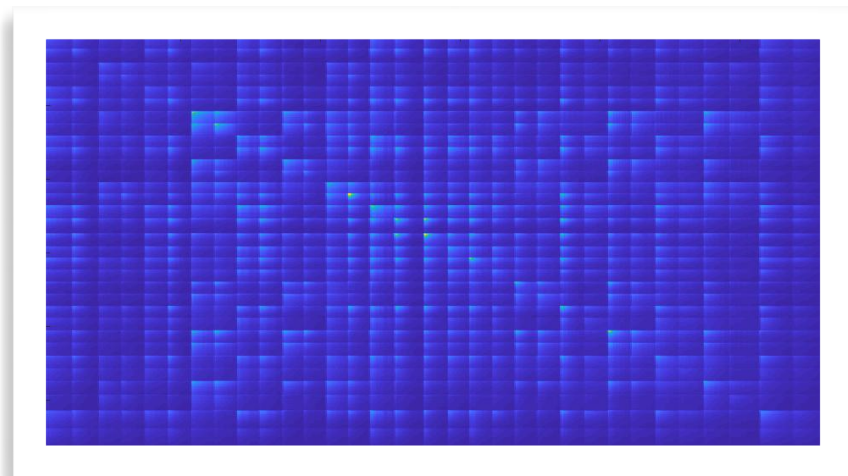
$$\phi_l^t = \begin{cases} 1, & \text{if } c_l \text{ is false for } \tilde{x}^t \\ 0, & \text{otherwise.} \end{cases}$$

This cost is used during the optimisation to implement a soft-constraint solver.



TEMPORAL MODELING

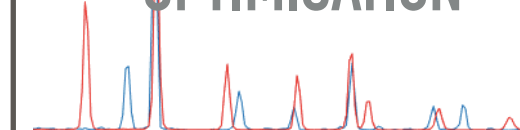
Target segmentation



For each segment....

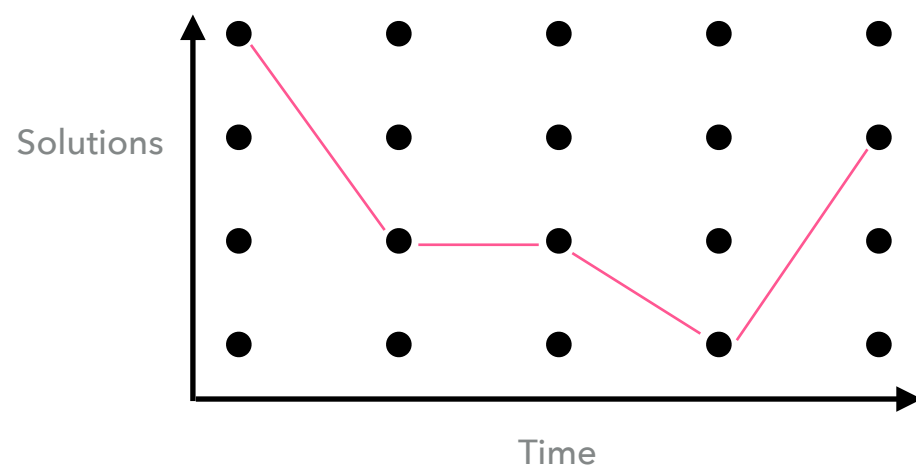
K-STOCHASTIC
PURSUIT

HIGH-DIMENSIONAL
OPTIMISATION



Joint-optimisation

Shortest-path continuation



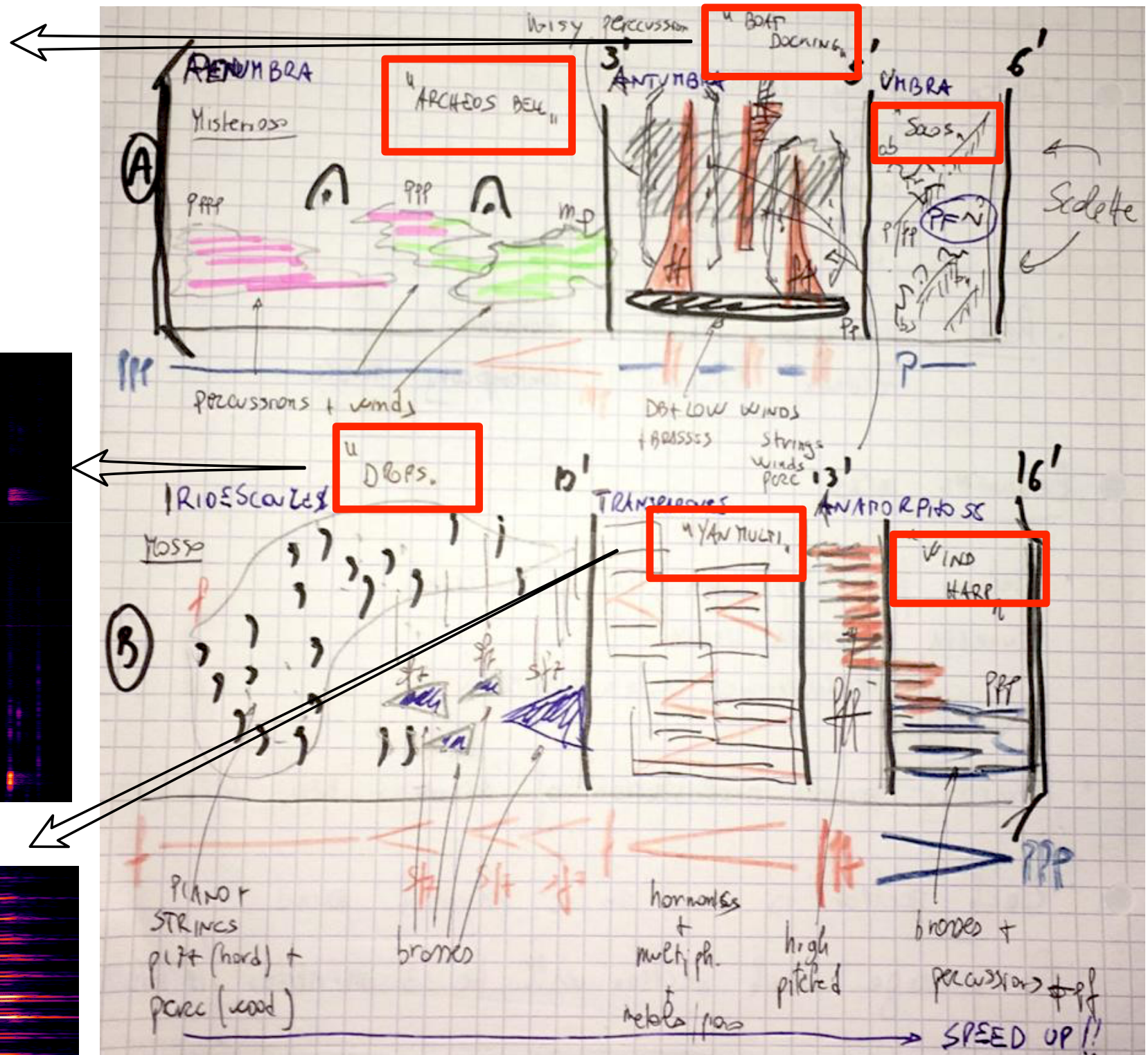
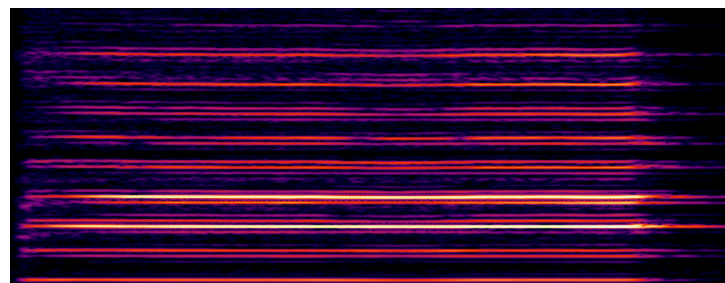
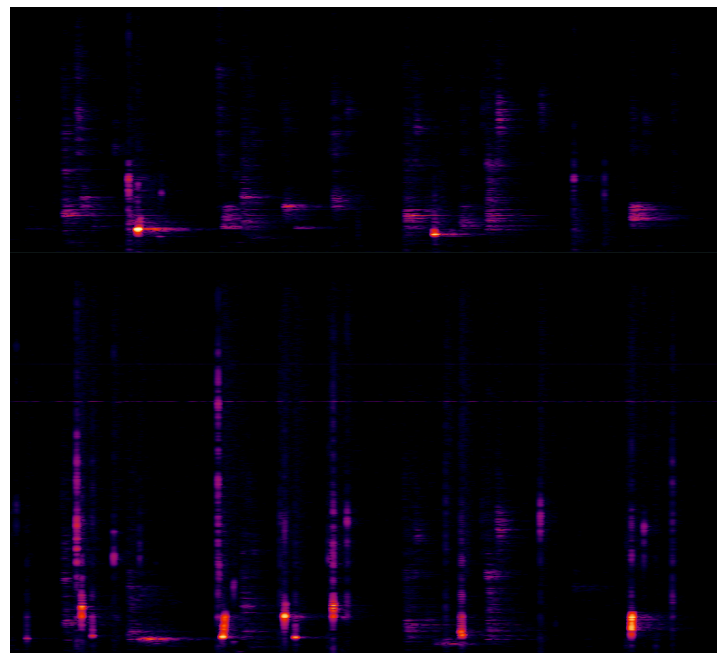
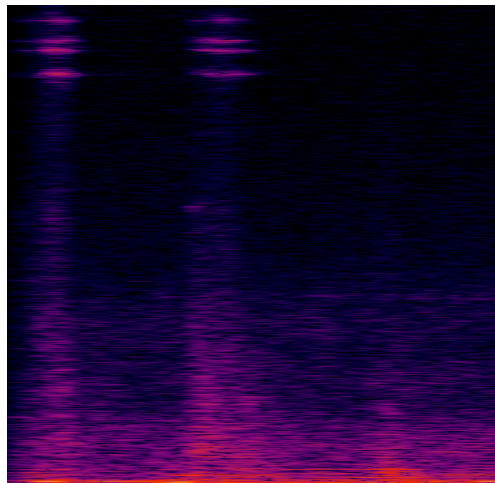
ORCHESTRAL HYSTERESIS (DOVE-TAILING)

$$\mathcal{M}(\hat{x}, \tilde{x}^t, i) = \int_{t-i}^t \mathcal{D}(\hat{x}^{t-i}, \tilde{x}^t) \tilde{x}^t dt \quad (24)$$

where $\hat{x}^{t-i}, \tilde{x}^t$ are the solution selected at a previous time point and the candidate combination at time t respectively, i is the memory window in time and $\mathcal{D}$ is defined as in equation [22](#).

A musical score for four instruments: Flute, Oboe, Violin, and Viola. The score is written in 3/8 time and features a key signature of one flat (B-flat). The Flute and Oboe parts are in the treble clef, while the Violin and Viola parts are in the bass clef. The Viola part is written in a lower register, using a C-clef. The score consists of four measures. In the first two measures, the Flute and Oboe are silent, while the Violin and Viola play a melodic line. In the third and fourth measures, the Flute and Oboe enter with a new melodic line, while the Violin and Viola continue their previous line. The Flute and Oboe parts are marked with a 'z' at the beginning of the third measure, indicating a change in the solution selected. The Viola part is marked with a 'z' at the beginning of the fourth measure, indicating a change in the candidate combination.

C. E. CELLA, STADES D'OMBRE, STADES DE LUMIÈRE (2018)



C. E. CELLA, STADES D'OMBRE, STADES DE LUMIÈRE (2018)

Target sound

Orchidea solution

Real orchestration

(longer excerpt)

Stades d'ombre, stades de lumière

For Eli

Carmine-Emanuele Cella (2018)

C Grave ♩ = 48

Flute
Oboe
Clarinet in B♭
Bassoon
Horn in F
Trumpet in B♭
Trombone
Percussion
Piano
Violin I
Violin II
Viola
Violoncello
Contrabass

harmonic tremolo
pp
mf senza crescere
ppp
p senza crescere
ppp
opening and closing with hand
ppp senza crescere
harmon mute
ppp
straight mute
ppp
irregular movements with superball
ppp
irregular movements with hands on the strings in the lower region
ppp
strike strings with hands
p
L.v.
L.v.

C Grave ♩ = 48

increasing vibrato speed
p espress.
on the bridge
ppp
mute
ppp
lead mute
increasing vibrato speed
on the bridge
ppp
pizz...
mp
increasing vibrato speed
mp espress.
ppp senza crescere

Proprietà per tutti i Paesi della SUGARMUSIC S.p.A. - Milano, Galleria del Corso, 4.
© Copyright 2018 by SUGARMUSIC S.p.A. - Milano.
Tutti i diritti riservati ai termini di legge - All rights reserved. International Copyright secured.

S. 15732 Z.

C. E. CELLA, STADES D'OMBRE, STADES DE LUMIÈRE (2018)

Target sound

Orchidea solution

Real orchestration
(longer excerpt)

FULL SCORE

Stades d'ombre, stades de lumière

For Eli

Carmine-Emanuele Cella (2018)

C Grave ♩ = 48

Flute

Oboe

Clarinet in B♭

Bassoon

Horn in F

Trumpet in B♭

Trombone

Percussion

Piano

Violin I

Violin II

Viola

Violoncello

Contrabass

harmonic tremolo

pp

mf senza crescere

ppp

mp

p senza crescere

ppp

opening and closing with hand

ppp senza crescere

harmon mute

ppp

straight mute

ppp

p

ppp

p

TAM TAM

irregular movements with superball

ppp

mp

irregular movements with hands on the strings in the lower region

strike strings with hands

ppp

p

ppp

p

increasing vibrato speed

p espress.

mp

on the bridge

ppp

mute

ppp

pp

ppp

pizz.

mp

lead mute

increasing vibrato speed

ppp

mp espress.

on the bridge

ppp

ppp senza crescere

Proprietà per tutti i Paesi della SUGARMUSIC S.p.A. - Milano, Galleria del Corso, 4.
© Copyright 2018 by SUGARMUSIC S.p.A. - Milano.
Tutti i diritti riservati a termini di legge - All rights reserved. International Copyright secured.

S. 15732 Z.

C. E. CELLA, STADES D'OMBRE, STADES DE LUMIÈRE (2018)

Target sound

Orchidea solution

Real orchestration

(longer excerpt)

Stades d'ombre, stades de lumière

For Eli

Carmine-Emanuele Cella (2018)

C Grave ♩ = 48

Flute
Oboe
Clarinet in B♭
Bassoon
Horn in F
Trumpet in B♭
Trombone
Percussion
Piano
Violin I
Violin II
Viola
Violoncello
Contrabass

harmonic tremolo
pp
mf senza crescere
ppp
p senza crescere
ppp
opening and closing with hand
ppp senza crescere
harmon mute
ppp
straight mute
ppp
irregular movements with superball
ppp
irregular movements with hands on the strings in the lower region
ppp
strike strings with hands
p
L.v.
L.v.

C Grave ♩ = 48

increasing vibrato speed
p espress.
on the bridge
ppp
mute
ppp
lead mute
increasing vibrato speed
on the bridge
ppp
pizz...
mp
mp espress.
ppp senza crescere

Proprietà per tutti i Paesi della SUGARMUSIC S.p.A. - Milano, Galleria del Corso, 4.
© Copyright 2018 by SUGARMUSIC S.p.A. - Milano.
Tutti i diritti riservati a termini di legge - All rights reserved. International Copyright secured.

S. 15732 Z.

C. E. CELLA, STADES D'OMBRE, STADES DE LUMIÈRE (2018)

Target sound

Orchidea solution

Real orchestration

(longer excerpt)

Stades d'ombre, stades de lumière

For Eli

Carmine-Emanuele Cella (2018)

C Grave ♩ = 48

Flute
Oboe
Clarinet in B♭
Bassoon
Horn in F
Trumpet in B♭
Trombone
Percussion
Piano
Violin I
Violin II
Viola
Violoncello
Contrabass

harmonic tremolo
pp
mf senza crescere
ppp
p senza crescere
ppp
opening and closing with hand
ppp senza crescere
harmon mute
ppp
straight mute
ppp
irregular movements with superball
ppp
irregular movements with hands on the strings in the lower region
ppp
strike strings with hands
p
L.v.
L.v.

C Grave ♩ = 48

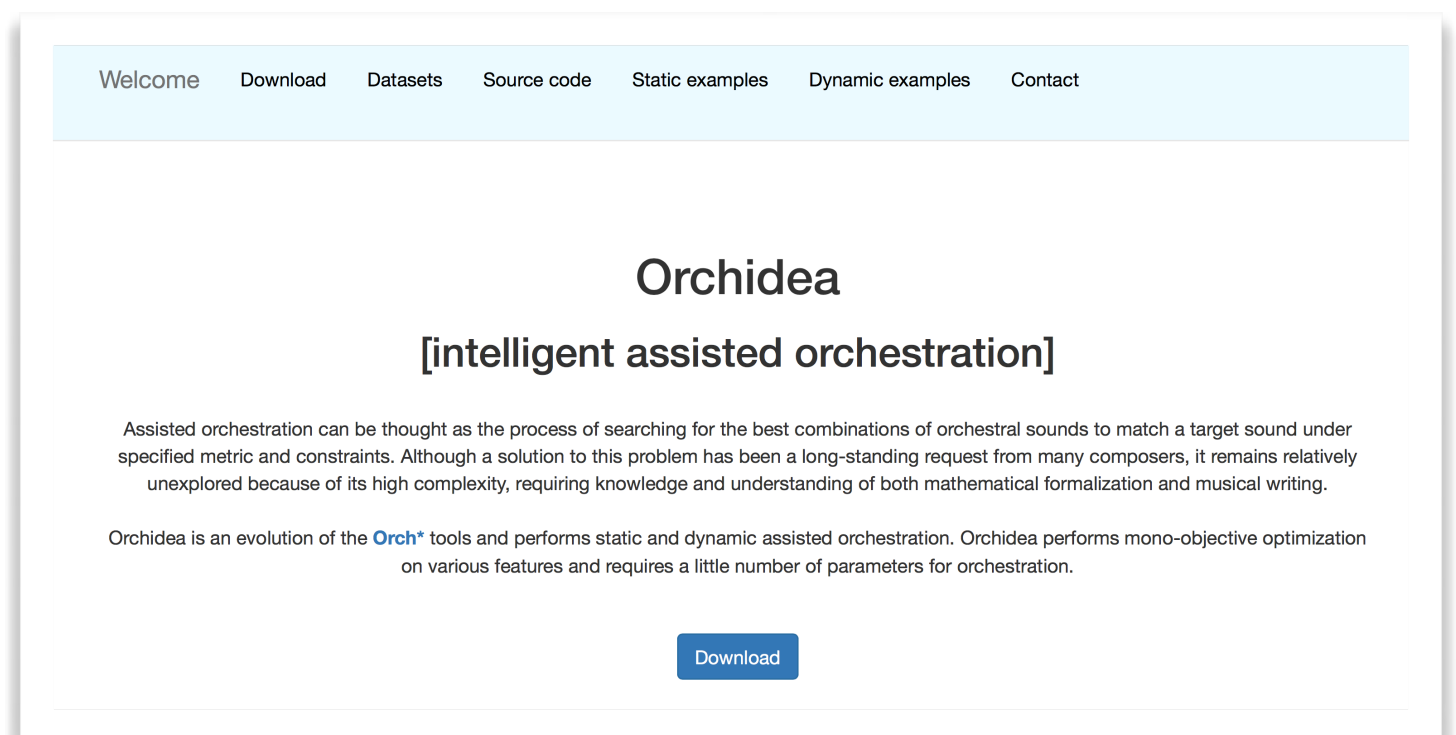
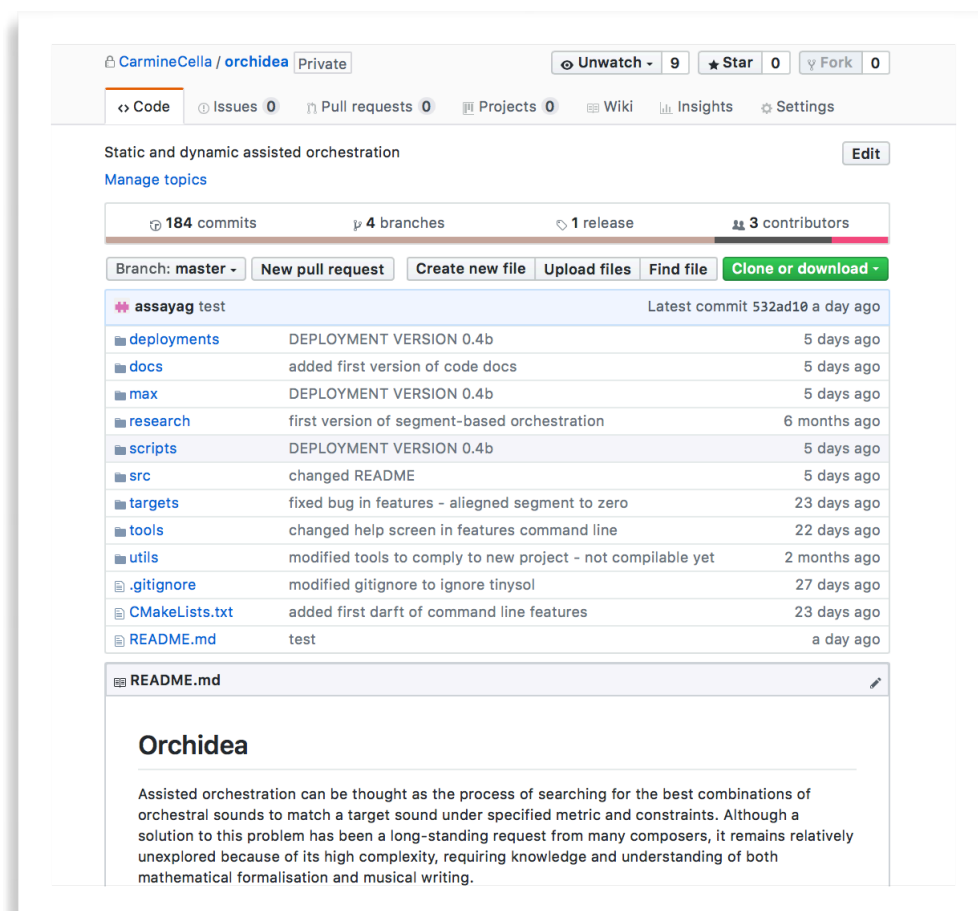
increasing vibrato speed
p espress.
on the bridge
ppp
mute
ppp
lead mute
increasing vibrato speed
on the bridge
ppp
pizz...
mp
mp espress.
ppp senza crescere

Proprietà per tutti i Paesi della SUGARMUSIC S.p.A. - Milano, Galleria del Corso, 4.
© Copyright 2018 by SUGARMUSIC S.p.A. - Milano.
Tutti i diritti riservati a termini di legge - All rights reserved. International Copyright secured.

S. 15732 Z.

ORCHIDEA TEMPLATE LIBRARY (C++) → OPEN SOURCE SOON!

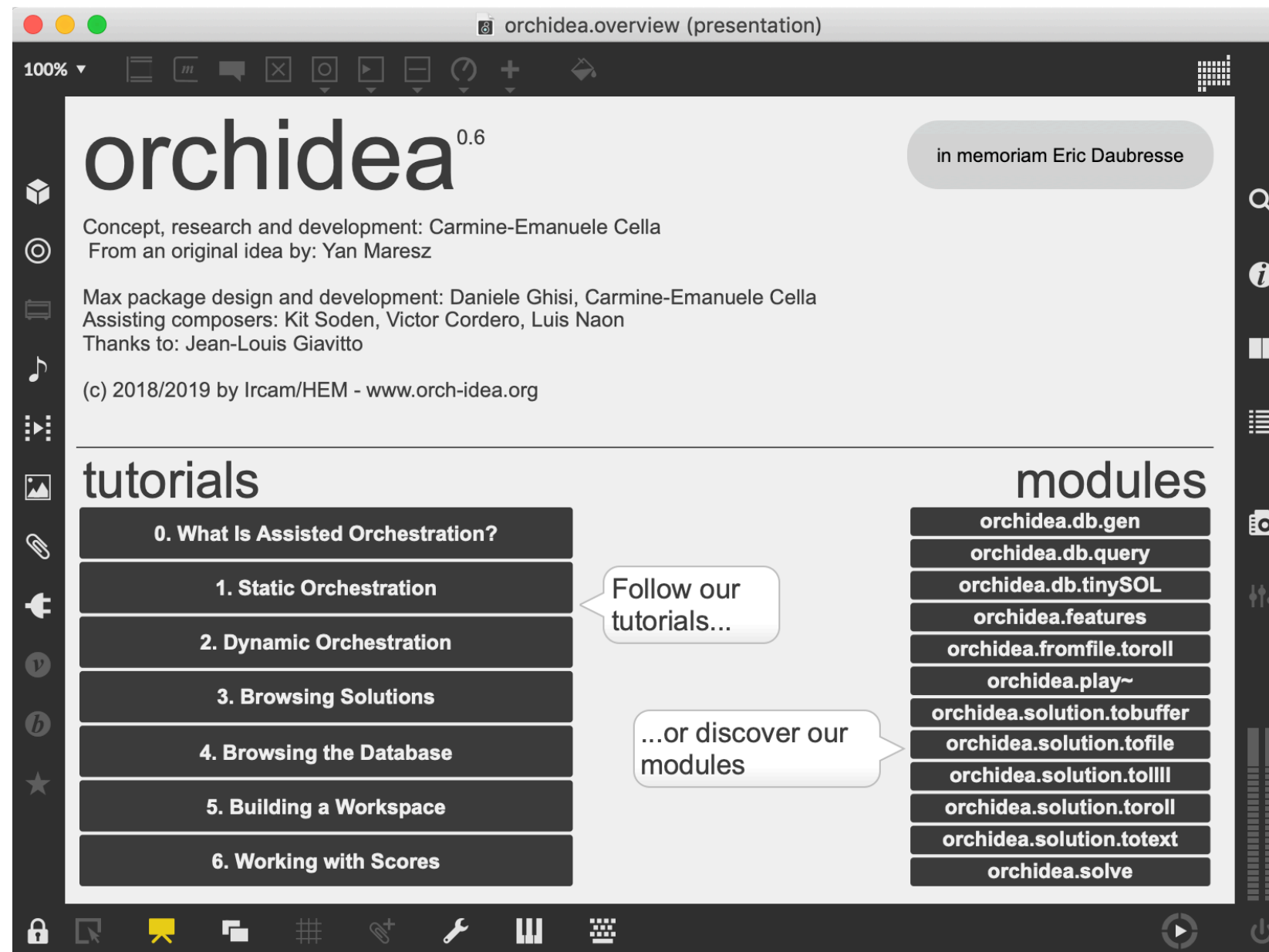
<https://github.com/CarmineCella/orchidea>



www.orch-idea.org

ORCHIDEA MAX PACKAGE

Carmine-Emanuele Cella, Daniele Ghisi



www.orch-idea.org

ORCHIDEA STANDALONE

Alessandro Petrolati, Daniele Ghisi, Carmine-Emanuele Cella

The screenshot displays the Orchidea Standalone software interface. On the left, a sidebar contains a 'Target Audio Source' section with a waveform and a 'Properties' section with buttons for 'Target', 'Solve From Script', 'Use', 'Orchestration', 'Quality', 'Initialization', 'Diversity', 'Sparsity', 'ANALYSIS', 'Partials Filtering', 'SEGMENTATION', 'Threshold', and 'Timegate'. The main area shows a musical score for various instruments: ASax, Acc, BTb, BTb+S, Bn, Cb, Cb+S, ClBb, Fl, and Gtr. The score is written in musical notation with various dynamics (pp, mf, ff, p, f) and articulations (pizz, stacc, key_click, etc.). The top of the interface features a timeline with markers at 0'00", 0'10", 0'20", 0'30", 0'40", and 0'50".

www.orch-idea.org